



Real Time Identification of Malicious Nodes in the Bitcoin Lightning Network using a Trust-based Algorithm

Neha Sadasivam

C23048790

September 2025

MSc Computing and IT Management

School of Computer Science and Informatics, Cardiff
University

A dissertation submitted in partial fulfilment of the requirements for
MSc in Computing and IT Management taught programme,
supervised by **Dr. Padraig Corcoran**

Acknowledgments

Firstly, I would like to express my deepest gratitude to my supervisor **Dr. Padraig Corcoran** for their invaluable guidance, expertise and encouragement throughout the course of this dissertation. Their insightful feedback and patient mentorship have been instrumental in shaping the direction and quality of this work. I am truly grateful for this time, dedication and effort they have invested in supporting my research.

I also extend my appreciation to the **School of Computer Science and Informatics at Cardiff University** for providing an intellectually stimulating environment and access to the necessary resources that made this research possible. The faculty and administrative staffs have always been supportive ensuring that the tools and assistance were provided when required.

I am equally grateful to my parents **Mr. Sadasivam** and **Mrs. Neena Sadasivam** who have been my pillar of support during this demanding period. Their encouragement and understanding have helped me maintain balance and perspective even during the most intense stages of this research. I would also like to mention my sisters **Mrs. Monica Sadasivam** and **Mrs. Swetha Lokesh**, and my cohort classmate **Kris Hu**, for their companionship, being a constant source of motivation.

Lastly, I would like to thank everyone including my well-wishers who in one way or another contributed to this dissertation. The support I received, whether through academic input or personal encouragement has always been invaluable and I am sincerely thankful.

Table of Contents

Table of Contents	2
List of Tables	4
List of Figures.....	4
Abstract.....	4
Chapter 1: Introduction	6
1.1 Overview of Denial-of-Service attacks in Cryptocurrency	6
1.2 Aims and Objectives.....	7
1.3 Research Contributions	7
Chapter 2: Background and Literature Review	8
2.1 Bitcoin and Lightning Network Overview	8
2.2 Routing, Liquidity and Timeouts in LN.....	8
2.3 Threat Model: DoS/Jamming Attacks in PCNs	9
2.4 Related Work and Reputation Models in P2P systems	10
2.5 Gaps in Current Research and Motivation for This Study.....	10

2.5.1 Connected Component Dynamics in Untrusted Subgraphs.....	12
Chapter 3: Problem Definition and Scope.....	13
3.1 Problem Statement.....	13
3.2 Unjamming Payments with Trust-Based Filtering	14
3.3 Adversary Model, Scope, Assumptions, and Constraints	14
3.4 Expected Outcomes and Research Boundaries	15
Chapter 4: Methodology.....	16
4.1 Overview of Methodological Approach.....	16
4.2 Simulation Design and Implementation.....	18
4.2.1 Rationale for using Simulation	18
4.2.2 Network Topology.....	18
4.2.3 Payment generation and Jamming behaviour.....	18
4.2.4 Data collection and Simulation settings	19
4.3 Trust Computation Using EigenTrust	19
4.4 Integration of Connected Components and Subgraph Isolation	20
4.5 Tools, Libraries, and Data Sources.....	21
4.6 Ethical Considerations.....	22
Chapter 5: System Design and Implementation	22
5.1 Graph Construction and Node Interaction Simulation	22
5.2 Detection of Malicious Nodes via Trust Scores	23
5.3 Generating and Filtering Untrusted Subgraphs	24
5.4 Connected Components for Source Isolation	25
5.5 Visualisation and Network Behaviour Modelling	25
5.5.1 Artefacts Produced	27
Chapter 6: Results.....	27
6.1 Overview and Evaluation Parameters.....	27
6.2 Eigen Trust Scores per Node and Visualisation of Trust Evolution	28
6.3 Connected Component Analysis and Timeout Delay Behaviour	30
6.4 Impact of Baseline Trust-based filtering on Routing Performance	32
6.5 Summary of Findings.....	33
Chapter 7: Analysis and Discussion	34
7.1 Evaluation of Research Objectives	34
7.2 Methodological Limitations.....	35
7.3 System Reliability and Performance Metrics	36
7.4 Implications for Lightning Network Design	37

Chapter 8: Conclusions	38
8.1 Summary of Contributions and Key Takeaways	38
8.2 Key Learnings and Practical Impact.....	38
8.3 Deployment Considerations and Adversary Adaptation.....	39
8.4 Future Enhancements	40
8.5 Final Remarks	41
Chapter 9: Reflection and Lessons Learned	42
9.1 Research Process Reflection	42
9.2 Technical growth and Domain Mastery	42
9.3 Project management and Decision-Making.....	43
9.4 Areas for Personal and Professional Development	43
References.....	44

List of Tables

Table 1 Comparison of existing Trust models in LN context	11
Table 2 Simulation parameters and output metrics	19
Table 3 Designated Node roles in test Network.....	22
Table 4 Summarization of interpreted result	31

List of Figures

Figure 1 Simple payment channel network with trusted nodes as circles and untrusted nodes as squares with node ID and message propagation illustrated through arrows ...	13
Figure 2 High level workflow of Trust computation.....	17
Figure 3 Pseudocode for graph construction and payment routing.....	23
Figure 4 Pseudocode for trust computation	24
Figure 5 Pseudo code for gossip simulation and origin detection.....	25
Figure 6 Trust score heatmap example	26
Figure 7 Overall simulation result.....	28
Figure 8 Trust score heatmap for our simulation run where each edge is coloured with its corresponding trust score	29
Figure 9 Untrusted connected component subgraph.....	30
Figure 10 Bar chart of cumulative delays per untrusted node	31
Figure 11 trust_scores.csv file	32
Figure 12 bad_payments_origin.csv file	32

Abstract

Bitcoin is the **Layer 1** network (base layer) of **Blockchain** technology. Layer 1 blockchains originally performs the core task of the network such as validating transactions, storing data

and are designed for security and decentralization rather than speed. But due to its low throughput, it can only handle few transactions per second with high on-chain transactions fees, making routine payments impractical. So, **Lightning Network** (LN) was introduced on top of Bitcoin as a **Layer 2** payment system, by enabling fast and low-cost transaction by conducting payments off-chain through a mesh of **bidirectional channels** i.e., it enables multi-hop mechanism where payments are not direct but instead are routed through intermediate peers and each peer has a finite amount of **liquidity** that is used to forward payments. But in terms of security and privacy, Lightning Network is trust minimised with **limited scalability** (Corcoran and Lewis 2024); the network's ability to handle payments quickly and affordably wherein intermediate nodes cannot thief funds, but the network can still be disrupted at the systems level. So, the liquidity can be targeted by a key threat known as **Denial-of-Service** (DoS) '**jamming**' attacks where attackers flood routes repeatedly with failing payments, locking **channel liquidity** where even legitimate peers can no longer forward payments and struggle to timeout and get through. Prior analyses show that even relatively small sets of hostile nodes can significantly hamper the connectivity and throughput in Lightning Network like topologies.

Although there are many ways to deal with these jamming attacks such as an **upfront fee** (Shikhelman and Tikhomirov, 2002) by imposing a small cost on every payment, this dissertation research explores a decentralized, **reputation management** approach in **peer-to-peer** (P2P) systems to strengthen Lightning Network against DoS attacks (jamming) without central authorities or protocol changes. To counteract these attacks, we adapt the concept of a **trust-based algorithm** where peers form local opinions from observed forwarding behaviour and aggregate them into transitive and global trust scores. Nodes that consistently enable successful payments are preferred; nodes correlated with repeated failures or long delays are isolating/deprioritised. To preserve privacy in payment channels, which is a huge concern, only signals about bad interactions are shared (no disclosure of successful payment flows), and the mechanism runs as an **overlay** policy, leaving Lightning Network's base protocol intact. Since real payment flow data is private and secure by design, we evaluate the approach via simulation of Lightning Network like graphs, modelling mixes of honest and malicious nodes, payment attempts and jamming behaviour. Using an iterative **trust computation** and simple threshold during route selection, our small to medium experiments reinforces faster identification of the misbehaving nodes and its origin, reduced jammed capacity and improved end to end success relative to baseline routing with modest overhead, forming a robust payment channel network, purely based on **Python** and its respective libraries and tools for this research implementation.

By integrating decentralized trust and reputation systems into path selection, can mitigate DoS impact correlating to maintaining privacy in Payment Channel Networks and we outline directions for larger-scale evaluation and practical deployment by scaling out the node simulations in real topologies and combining trust with light cost signals.

Chapter 1: Introduction

1.1 Overview of Denial-of-Service attacks in Cryptocurrency

Bitcoin has established itself as the most pioneering **cryptocurrency** till date since its inception in 2009 (Nakamoto, 2008). Being a decentralized system, it enables secure transfer of digital assets between participants without relying on central authorities. While revolutionary, Bitcoin's underlying blockchain protocol suffers from inherent scalability limitations, processing roughly **seven transactions per second** with each transaction requiring about **ten minutes** for confirmation. Such performance is unreliable for high-frequency, low-latency payment scenarios such as purchasing a coffee or conducting microtransactions.

To overcome these challenges, second-layer payment protocols have emerged, the most prominent being the **Lightning Network (LN)**. Lightning Network enables participants to open bidirectional payment channels, conduct numerous instant low-cost transactions with negligible fees and only commit the final net settlement to the blockchain, supporting near real-time payments. This approach drastically reduces on chain congestion in Payment channel network (PCN), improves **privacy** because a given channel only corresponds to two blockchain transactions, one to **open** the channel and one to **close** the channel. So, the details of the channel transactions are only known to the channel participants and remain private unlike blockchain transactions that are broadcast to all peers.

Nevertheless, Lightning Network is susceptible to system-level attacks that can payment routing failures without breaching cryptographic security. One of the most damaging forms is the **Denial-of-Service (DoS)** which is basically **Jamming attack** in Blockchain, where malicious nodes deliberately send payments designed to fail (fake payments) or ties up **channel liquidity** (Seres et al., 2020; Pérez-Solà et al., 2020; Shikhelman and Tikhomirov, 2022); the ease and speed with which cryptocurrency can be bought or sold without affecting its prices. Even a small fraction of malicious participants can severely degrade Lightning Network's routing success rate and becomes a major trade-off for privacy.

In the context of cryptocurrency, DoS attacks can have multiple damaging effects such as:

- **Liquidity Locking:** Attackers occupy available channel capacity with bogus transactions preventing valid payments from being routed.
- **Routing Disruption:** By continuously forcing repeated failures, attackers can fragment the network's routing graph, isolating honest nodes and create bottlenecks.
- **Increased Transaction Costs:** Prolonged traffic forces honest users to route through less optimal, higher fee channels or retry payments multiple times, increasing the overall transaction costs.
- **Reduced Trust and Adoption:** Persistent service degradation sabotages user confidence, discouraging adoption of Lightning Network as a reliable payment solution.

- **Indirect Profit for Attackers** (which is a niche): In competitive routing scenarios, malicious hackers or attackers may use DoS attacks to divert traffic through their own nodes, increasing fee revenue.

The motivation for this research stems from the need to fortify Lightning Network against such attacks without undermining its core principles for decentralization, privacy and low fees. In real-world terms, attacks on cryptocurrency infrastructure have caused major financial losses. For example, the **2025 crypto heist in North Korea** reported by the BBC (BBC News, 2025), involved coordinated exploitation of vulnerabilities to steal millions of digital tokens. Although this specific incident was not a DoS attack, it highlights and infers that cryptocurrency systems being lucrative are fragile and vulnerable, making resilience measures critical for their long-term viability.

1.2 Aims and Objectives

The primary aim of this dissertation is to design and evaluate a decentralized trust and reputation-based mechanism to alleviate jamming attacks in the Bitcoin Lightning Network, enhancing its resilience without altering its core protocol which is the user privacy. Leveraging the principles of a **peer-to-peer** trust based **EigenTrust algorithm** (Kamvar, Schlosser and Garcia-Molina, 2003), an algorithm based on Google's PageRank algorithm, the approach focuses on identifying and isolating **malicious nodes** based on historical payment forwarding behaviour.

To achieve this aim, this dissertation pursues the following objectives:

- **Objective 1:** Analyse the vulnerabilities of Lightning Network's payment channel network to Jamming attacks and evaluate their impact on routing efficiency and scalability.
- **Objective 2:** Design a simulation framework using **Python (Matplotlib & NumPy)** and **NetworkX**, a software library for graph modelling (Hagberg, Schult and Swart, 2008) and connected components in Lightning Network like topologies, both honest and malicious nodes, payment routing, failure propagation and gossip simulation.
- **Objective 3:** Implement a global trust computation module to assign dynamic local trust scores, prioritising high reputation nodes during path selection, based on **EigenTrust** due to its proven reliability and robustness to evaluate reputation in many existing application domains, making it a stronger candidate for this research.
- **Objective 4:** Simulate and assess the efficacy of the trust-based mechanism in jamming impacts, as a user-based evaluation cannot practically mitigate real payments making it infeasible to observe and scrutinize malicious behaviour.

1.3 Research Contributions

This dissertation research contributes to the evolving field of **security** and **trust management** within Payment Channel Networks, especially the Bitcoin Lightning Network. Firstly, it introduced decentralized reputation-based overlay mechanism that enhances

Lightning Network path selection by ensuring that nodes preferentially route through high trust peers, while maintaining the payment protocol's privacy guarantees. Secondly, a novel simulation framework is developed to model Lightning Network like topologies under hostile conditions, capturing Denial-of-Service jamming behaviours and supporting the analysis of mitigation strategies in a controlled environment. A significant innovation lies in the adaptation of EigenTrust algorithm, originally designed for peer-to-peer file sharing networks to the context of payment routing, with a customization of domain-specific variables such as **channel liquidity exhaustion** and **timeout-based failures**. Having its own challenges and limitations, finally this work provides empirical evidence that reinforces trust-based decision-making into route selection, leading to measurable reductions in failed payments and improved network resilience. These findings are incorporated through scenarios-based experiments, including **subgraph analysis** of malicious components and peer-to-peer message propagation, offering practical insight into real-time attack mitigation strategy in decentralized financial systems.

Chapter 2: Background and Literature Review

2.1 Bitcoin and Lightning Network Overview

Bitcoin, introduced by **Satoshi Nakamoto** in **2008**, is a revolutionized digital currency that allows decentralized peer-to-peer transactions without a central authority (Nakamoto, 2008). It employs a proof-of-work (PoW) consensus mechanism and maintains a distributed public ledger **Blockchain**, that records all validated transactions. While Bitcoin ensures high security and transparency, it suffers from significant delays and scalability limitations, processing only around **7 transactions per second (TPS)** with each block confirmation taking approximately 10 minutes. This low throughput and high latency make it ineligible for every day and time-sensitive transactions.

To address these limitations, second-layer scaling solutions such as Lightning Network have been proposed. Lightning Network channel-based structure allows two parties to open bidirectional payment channel without revealing the transaction details and conduct multiple off-chain transactions. For example, a peer Alice forwards a payment to another peer Bob, who then forwards the payment to yet another peer Charlie. In this case, Alice and Charlie do not share a channel and payment is sent through an intermediate peer Bob, which explains the **multi-hop routing mechanism**. Only the final state of the channel is broadcast to the blockchain, significantly reducing network congestion and fees. It also enables faster transaction processing and settlement, better privacy, improved scalability, potentially supporting millions of TPS and making it a stronger candidate for supporting high-volume, low-latency payment demands in cryptocurrency domain.

2.2 Routing, Liquidity and Timeouts in LN

Lightning network's payment **routing** operating similarly like traditional packet-switching networks, relies in finding a path through the payment channel network (PCN) from sender to receiver. In practice, the PCN is treated as an unweighted graph indicating the presence of a

connection i.e., edges. These implementations are based on EigenTrust algorithm, a graph-based algorithm that computes trust scores for those peers based on the outcomes of the routed payments in an unweighted graph. It is typically based on Google's PageRank algorithm where it maintains rankings of peers that they have successfully routed through to improve path finding.

A payment is broken into smaller **Hashed Timelock Contracts** (HTLCs) and are routed through several intermediary nodes. Each channel must have adequate liquidity to forward the payment and a time constraint i.e., a **timeout** to be imposed to ensure **atomicity**; a guarantee that the payment either succeeds entirely across all hops or fails without any partial transfer. This property is significant and crucial to prevent scenarios where funds are locked up in intermediate nodes without reaching the final recipient node.

Routing success is contingent on factors such as channel capacity, fee policy, route length and node reliability. Failed payments may occur due to unavailable liquidity, outdated network views or malicious interference. Each failed payment in the given timeframe incurs timeouts, increasing transaction latency and reducing overall network performance.

This behaviour known as **channel jamming** exploits this timeout mechanism by sending repeated payments designed to fail, locking up liquidity and rendering routes unusable. Attackers may open multiple channels or flood existing ones with HTLCs that never complete, causing cascading failures across the Payment channel network. This deteriorates the core promise of Lightning Network-fast and cheap payments.

2.3 Threat Model: DoS/Jamming Attacks in PCNs

Denial-of-Service (DoS) attacks are a class of adversarial behaviours aimed at degrading a system's availability and performance. In the Lightning Network's context, DoS manifests as channel jamming where attackers intentionally clog payment routes by sending HTLCs that either never resolve or timeout deliberately. This consumes channel resources causing legitimate payments to fail.

Channel jamming is particularly insidious because it requires no breach of **cryptographic primitives**. Instead, it exploits the system's reliability on trustless forwarding and liquidity constraints. Even a small set of coordinated malicious nodes can inflict disproportionate harm by selecting key routing paths, thereby maximizing the disruption level.

To neutralize jamming attacks, several countermeasures have been proposed (Pérez-Solà et al., 2020; Shikhelman and Tikhomirov, 2022; Tikhomirov et al., 2020). These including introducing **upfront fees**, where every forwarded payment successful or not, incurs a cost, thereby disincentivizing spamming. Another method is **adaptive reputation systems** that track nodal behaviour over a period and prioritize routing through valid peers. However, each approach has trade-offs in terms of privacy, decentralization and ease of practical deployment.

2.4 Related Work and Reputation Models in P2P systems

Reputation-based systems have long been proposed as effective defences against misbehaviour in distributed environments. For example, the World Wide Web’s security relies on **central certificate authorities** (Transport Layer Security) to vouch for websites. In contrast, peer-to-peer (P2P) networks, where central authorities are absent, necessitates a **decentralized system** where trust must emerge organically based on participant behaviour. This bridges the gap between centralised trust and the decentralized ethos of Bitcoin Lightning Network. One of the most influential models is **EigenTrust** (Kamvar et al., 2003), originally designed for file-sharing networks, computes a global trust value by aggregating local feedback and transitive trust paths. Nodes that consistently behave well accumulate high scores, while unreliable peers are deprioritized.

In the context of payment channel networks, trust models are less common due to the privacy-preserving nature of transactions and the expectation of trustlessness. Nonetheless, recent proposals suggest adapting these models to enhance network resilience. For instance, (Fan, Chen and Liu, 2012) proposed **EigenTrust++** to extend reputation mechanisms to complex distributed systems. Meanwhile, (Guasoni, Minca and Sedunov, 2024) highlights the challenges of routing under adversarial settings and suggest incorporating behaviour aware heuristics to improve path selection.

Another notable contribution comes from (Aina and Litchfield, 2012) who explored **trust propagation mechanisms** that share only negative feedback to maintain privacy. This aligns with Lightning’s ethos, where only signals about failed or suspicious transactions are propagated, minimizing data leakage. Similarly, (Shikhelman and Tikhomirov, 2022) employs a **localized reputation** approach to handle malicious nodes with an unconditional fee based on repeated failure patterns.

These studies reveal a growing unanimity while the Lightning protocol itself remains unchanged, integrating a decentralized reputation overlay can significantly improve robustness. By learning from P2P trust management, Lightning Network can distinguish between benign and malignant routing behaviour in a privacy-protection manner.

2.5 Gaps in Current Research and Motivation for This Study

While existing literature recognizes the severity of jamming attacks and explores various countermeasures, several critical gaps persist. First, many reputation-based systems are only designed for **permissioned** or **localized peer-to-peer systems** for static or file-sharing networks where node participation is restricted. These systems are not **globalized** i.e., not inherently scalable or adaptable to open decentralized networks like the Lightning Network, where nodes can freely join and leave and where trust must be computed in a global and dynamic manner. Applying them to payment systems introduces unique constraints such as liquidity management, atomicity and time sensitivity which are not satisfactorily addressed in earlier works. Second, most traditional trust models either rely on local, non-collaborative observations or require extensive data sharing, which could violate user privacy. Our approach seeks a middle ground to build a robust system which is viable, where nodes selectively share

only negative interaction signals i.e., failed payments routes, while successful transactions remain private. This ensures privacy is intact while still enabling a global picture of malicious behaviour. This paved way for the use of **EigenTrust algorithm** due to its proven reliability in distributed peer-to-peer systems. Originally designed for file-sharing networks, it aggregates a local trust relationship into a global reputation score without a central authority, aligning closely with Lightning Network’s decentralized architecture. Third, few implementations offer empirical validation in Lightning Network like topologies. Our simulation framework not only models realistic Lightning Network behaviour but also evaluates how **reputation scores** evolve over time and impact routing efficiency.

Finally, there’s limited research on the dynamics of **NetworkX’s connected component** within subgraphs of untrusted nodes. By isolating these clusters and simulating peer-to-peer message interaction, our work identifies the origin of malicious activity and suggest counter routes to improve the network’s robustness.

The below **Table 1** gives a precise comparison of existing approach versus the proposed approach:

Model	Refer ence Paper	Type of Trust	Information sharing	Privacy considerations	Application Domain	Limitations
EigenTrust	Kamvar et al., 2003	Global (transitive)	Local aggregation into global	No explicit privacy guarantees	P2P File Sharing	Assumes availability of full transaction history
EigenTrust++	Fan et al., 2012	Global with local boost	Local and limited global	Attempts to reduce trust inflation	Wireless Ad Hoc & IoT	Vulnerable to group-based collusion
Personalized EigenTrust	Aina and Litchfield, 2012	Decentralized trust	Local only	High privacy, but limited scope	P2P Networks	No global trust propagation
Unjamming Lightning	Shikhelman and Tikhomirov, 2022	Path-based economic scoring	No reputation sharing	Payment data remains private	Lightning Network	High complexity, assumes rational agents only
Proposed approach	-	Global trust via simulation	Only shares bad interactions	Preserves successful transactions’ privacy	Lightning Network Simulation	Still requires real-world integration for deployment testing

Table 1 Comparison of existing Trust models in LN context

2.5.1 Connected Component Dynamics in Untrusted Subgraphs

A **connected component** in **graph theory** is a set of nodes that can reach each other via some path. Payments in the Lightning Network can only be routed within a single connected component; if two nodes lie in different components, no route exists between them. (add reference 8) So, the Lightning Network routing can be modelled as an undirected graph $G = (V, E)$, where each vertex $v \in V$ represents a Lightning peer and each edge $(u, v) \in E$ represents a bidirectional payment channel. Such graph models are standard in network science and allows us to analyse routing and connectivity. Existing research recognizes that jamming attacks deliberately lock channel liquidity and degrade the Lightning Network's routing capability. (Pérez-Solà et al., 2020) succinctly shows how attackers flood channels with payments that are curated to fail, forcing honest users' transactions to timeout. (Shikhelman and Tikhomirov, 2022) propose economic mitigations such as unconditional base fees, while (Seres et al., 2020) analyse the topology of Lightning Network to measure its resilience. However, none of these approaches examine how untrusted nodes interact with each other once they have been identified and isolated from the trusted set. In other words, there is little understanding of the internal dynamics of the '**untrusted subgraph**'. Although (Guasoni et al., 2024) observes that any node can pay any other node within the same connected component of the Lightning Network, they do not analyse the behaviour of these components when populated with low trust nodes.

Our work fills the gap by decomposing the untrusted subgraph into connected components, a cluster of nodes consisting of untrusted nodes that are mutually reachable. Analysing these clusters answer two critical questions:

- (1) how malicious behaviour propagates within an untrusted cluster?
- (2) which nodes most often act as origin of failed payments?

To illustrate these concepts, **Figure 1** depicts a simple payment channel network of six peers modelled as the $G = (V, E)$ graph. Nodes are Lightning Network peers and edges are channels. This graph framework underpins the trust and component analysis used later in this research.

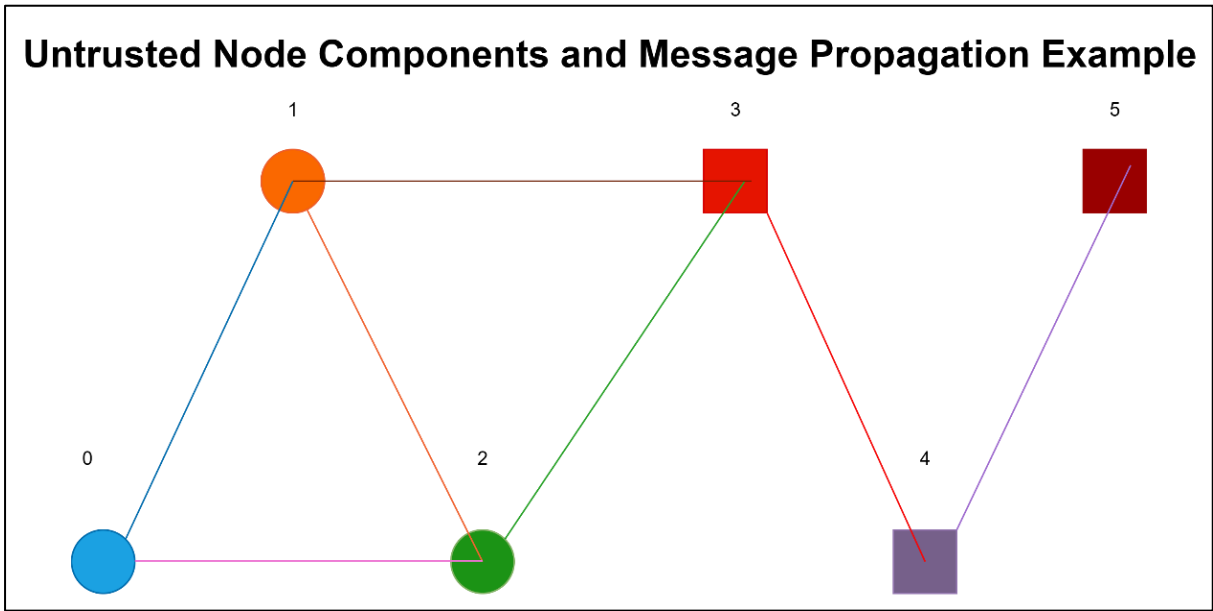


Figure 1. Simple payment channel network with trusted nodes as circles and untrusted nodes as squares with node ID and message propagation illustrated through arrows

In the above figure, trusted nodes are depicted as circles and untrusted as squares with random colours and Node ID allocated for each node. Nodes 3,4 and 5 are low trust nodes that form a connected component, while nodes 0-2 are trusted. Arrows indicate the direction of message propagation with respective node colours from left to right from each node. By simulating gossip among the untrusted nodes, we can observe that node 3 is responsible for initiating most failed payments (designated malicious node), and that nodes 4 and 5 merely relay those failures. Once node 3 is isolated or deprioritised in route selection, the cluster stops disrupting payments and honest congestion can be re-routed through trusted peers. This visual clustering example demonstrates the dual strategy of **trust-based scoring** and **component level analysis**. Trust metrics allow us flag individual peers who frequently trigger failures while component analysis lets us cordon off the entire cluster in which those peers operate. Together these approaches provide actionable intelligence, i.e., isolating the entire cluster and focusing on their ‘patient zero’, prioritising both identification of the originating node and containment of malicious activity. This combination abruptly reduces the impact of jamming without needing to penalise every untrusted node, which in particular has not been applied to payment channel networks in previous research.

Chapter 3: Problem Definition and Scope

3.1 Problem Statement

The Lightning Network (Layer 2) inherits Bitcoin’s (Layer 1) trust minimised decentralized model but introduces its own reliability challenges. As Lightning Network routes off-chain

transactions through a series of bidirectional channels, most users must send their payments via intermediaries because they do not have a direct connection (i.e., share a common channel). Each channel has a finite number of **HTLC slots** equals to the number of concurrent payments it can process and a limited liquidity balance wherein the resources are temporarily locked while a payment is in transit. In recent years, researchers have shown that adversaries target these mechanics to mount Denial-of-Service (DoS) ‘jamming’ attacks. A jamming attacker repeatedly bombards a target channel or sets of channels with payments that are purposely curated to fail or to stay unresolved until expiry, consuming the corresponding liquidity or exhausting the slots. Since a single malicious node can jam an entire payment route, even a small proportion of malevolent participants can significantly reduce Lightning Network’s payment success rate and sabotage user trust. Our problem, therefore, is to develop a real-time method to detect these misbehaving nodes and prevent them from degrading the network performance without sacrificing Lightning Network’s decentralized protocol and structure or violating privacy of legitimate users.

3.2 Unjamming Payments with Trust-Based Filtering

In this study, we propose a trust-based filtering mechanism to mitigate or unjam jamming attacks. The key idea is to compute a dynamic reputation score for each node based on its observed payment forwarding behaviour. When a payment fails or times out, the node on which it failed records a negative interaction. Nodes then share only these negative signals; they do not share information about the successful payments to preserve privacy of the honest transactions while still signalling bad behaviour, as bad payments are a minority (forming only a small fraction of total traffic) and therefore this approach only generates a small additional communication overhead. Furthermore, an adapted EigenTrust algorithm prioritises routes through high trust peers and deprioritises low trust peers. This approach directly addresses DoS jamming where malicious nodes that initiate these failed payments quickly accrue low trust and are avoided by honest route planners.

Jamming attacks can come in two main variants. **Liquidity jamming** (often called the loop attack) involves sending a relatively large payment to oneself across many intermediate channels and delaying settlement or failure; it locks up multiples of the attacker’s own capital along the route (add reference). **HTLC slot jamming** or **HTLC jamming** uses a flurry of tiny payments to exhaust the limited number of in-flight HTLC slots in each channel (add reference). Our trust-based filtering agnostic to these variants wherein any node that frequently causes payments to fail or timeout (whether by occupying liquidity or by filling slots) receives a lower trust score. Practically this means that when multiple payments are attempted, the algorithm quickly identifies and isolates the nodes responsible for delays. Primarily, each HTLC that is in progress occupies one of the finite slots for a set period; our approach mitigates the problem by preventing malicious nodes from accumulating failures that block slots. (Bitcoin Optech, n.d.)

3.3 Adversary Model, Scope, Assumptions, and Constraints

We model adversaries as Lightning nodes that actively attempt to reduce the success rate of honest payments through jamming. Attackers can open channels and create payment routes

like any other node; they may collude and can decide how many HTLCs to send, how much liquidity to lock and when to cancel their payments. Honest nodes follow the protocol and report failed or delayed transactions. We assume that the majority of nodes are honest and that the network graph remains connected. Our trust algorithm relies on aggregating feedback from many sources to compute global scores. Our study does not consider Sybil attacks (where an adversary creates many false identities) or balance probing attacks (gathering information of a target network from an external source network); we assume that Lightning Network’s underlying cryptography remains secure and that adversaries cannot steal funds. All data shared between nodes are negative interaction signals where nodes do not reveal channel balances or successful payments.

The scope of this work is limited to simulation and the analysis of these simulations. We implement the payment channel network as an unweighted graph and simulate payment attempts between randomly selected pairs. We measure performance metrics such as trust convergence, detection latency, payment success rate and the size of untrusted components. We do not modify the protocol of Lightning Network or handle on-chain operations. Instead, we design an overlay mechanism that could be incorporated into existing Lightning Network node software without modifying Lightning Network’s protocol. Our simulations use a relatively small network with static channels to focus on illustrating trust dynamics highlighting its non-invasive nature whereby real Lightning Network topologies are larger and more dynamic, we discuss these limitations in Chapter 7.

3.4 Expected Outcomes and Research Boundaries

We expect that applying trust-based filtering to jamming attacks will yield three key outcomes. First, malicious nodes will be rapidly assigned low trust scores, allowing honest peers to route around them. Second, payment success rates and available liquidity will improve because fewer channels are jammed. Third, untrusted nodes will form identifiable connected components that can be separated with minimal collateral damage to honest peers. By analysing message propagation within these components, we can identify the nodes that originate most failures and recommend alternative routes. Together, these results should demonstrate that privacy-preserving reputation system can essentially enhance Lightning Network’s resilience against DoS attacks without centralisation or fee charges.

There are boundaries to what this research can accomplish. Our simulation does not capture all aspects of a live Lightning Network; real routing algorithms are more complex, channels are constantly opened and closed, and economic incentives may change attackers’ behaviour. We do not propose new economic mechanisms such as upfront fees, although we acknowledge these as complementary defences. We also do not evaluate the system under large-scale Sybil attacks or consider the computational overhead of trust calculation in very large networks. Finally, while we discuss both liquidity jamming and HTLC slot jamming, our mitigation strategy focuses on routing behaviour and does not address other potential attack surfaces such as probing or channel exhaustion by force-close attacks. (Bitcoin Optech, n.d.)

Chapter 4: Methodology

4.1 Overview of Methodological Approach

This chapter outlines the methodological framework used to design, implement and evaluate a trust-based defence against jamming attacks in Lightning Network. The overall proposed approach combines **simulation**, **peer-to-peer reputation calculation** and **graph theoretic analysis**. Because real payment data are not publicly accessible i.e., Lightning payments are private and only channel opening and closing transactions are recorded on-chain, making it infeasible to observe the nature of actual malicious identities in a live environment, we build a synthetic Lightning-like payment network and simulate payment routing under adversarial conditions. Ethical and legal consideration are also preclude deliberately jamming the live network. Using this simulation, our methodology proceeds through several stages such as:

- (1) construction a Lightning-like payment channel graph.
- (2) simulating x number of payments under honest and malicious behaviour.
- (3) computing local and global trust scores via the EigenTrust algorithm.
- (4) identifying untrusted nodes and isolating them into connected components.
- (5) analysis message propagation within these components to pinpoint the **origin of failures**.

Figure 2 provide the methodological workflow for this research.

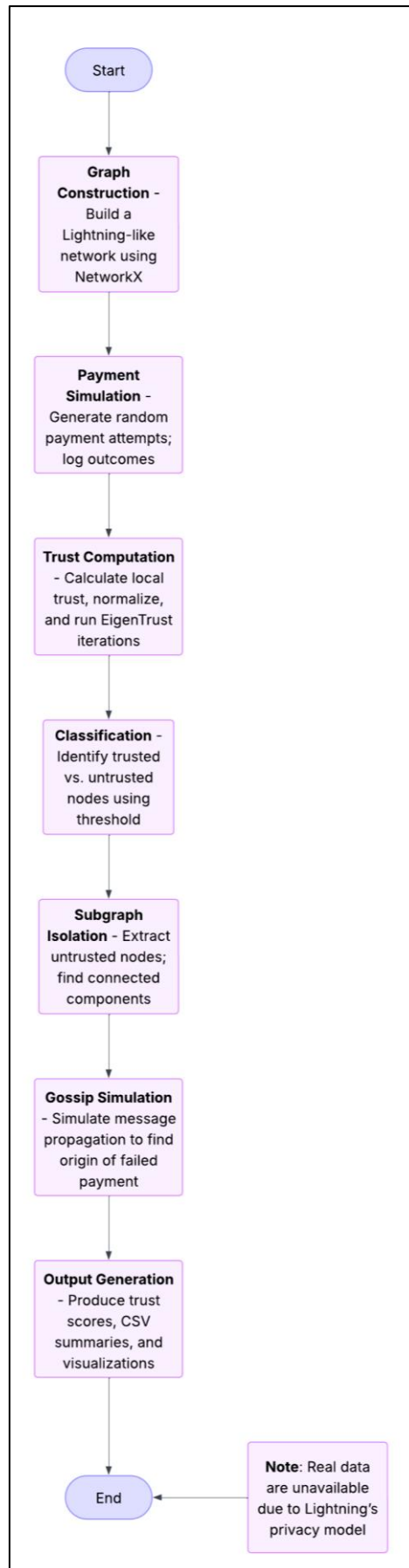


Figure 2. High level workflow of Trust computation

4.2 Simulation Design and Implementation

4.2.1 Rationale for using Simulation

A simulation-based approach is essential and effective for evaluating DoS attack mitigation in Lightning network. Since Lightning Network maximises user privacy wherein payment paths, identities of each peer (node) and failure events are not publicly observable, no global ledger records failed payments or routing paths. Moreover, it is decentralized and permissionless where there is no central authority to share data. So, a user-based evaluation would be anecdotal or biased. Attempting to jam the live network would risk locking real users' funds and violate privacy and network terms. Simulations, in contrast, mimics this real-time jamming, providing full control over node behaviours, network topology, payment volume and timeout thresholds. This enables reproducible experiments under varying circumstances and allows stress testing scenarios such as large-scale jamming or global trust decay, that would be impractical and unethical on the actual network. Many prior studies of Lightning Network security and trust are likewise based on simulation, so our methodology remains consistent with the existing literature.

4.2.2 Network Topology

Our simulation starts by defining an unweighted and undirected graph $G = (V, E)$, where each node $v \in V$ represents a Lightning node and each edge $(u, v) \in E$ represents a bidirectional payment channel. In our baseline scenario $N = 5$ nodes 0-2 are trusted and nodes 3-4-5 are untrusted, forming a malicious cluster. This toy network suffices the illustration of core ideas reputation computation, jamming attacks and component analysis. The underlying network structure is tailored and generated using NetworkX graph primitives and we assign a capacity (liquidity) and a fixed number of HTLC slots to each channel. (Poon and Dryja, 2016)

Note: Here V and N are the same where V denotes the number of vertices (nodes) in a graph according to graph theory. In our simulation the variable N is used for the same purpose where it stores the number of nodes to initialize arrays and loops through nodes. Our python script uses N to initialize trust vectors and matrices and V for elaboration purpose. So N in shorthand is just V .

4.2.3 Payment generation and Jamming behaviour

To model typical Lightning Network usage, the simulator randomly selects sender-receiver pairs and attempts to route payments along the shortest path (NetworkX is employed for computation). Since our payment channel network was modelled as an **unweighted graph**, the function `nx.shortest_path()` internally uses a **breadth-first search (BFS)** strategy rather than **Dijkstra's weighted shortest path algorithm**. This ensured that each simulated payment followed the minimum hop path. Each simulated payment is represented as a **Hashed Timelock Contract (HTLC)** that occupies exactly one slot on each channel along its path. We set a global timeout mechanism to emulate HTLC expiry; if a payment does not complete before the timeout, it fails, and all locked liquidity is returned to the channel. Honest nodes always forward payments immediately whereas the designated malicious node

(node 3 in our simulation) purposely delays or cancels payments at the last possible moment. Such behaviour imitates both **liquidity jamming** and **HTLC jamming** where an attacker either sends one payment across many channels to lock large amount of funds or floods channels with small payments to exhaust the limited number of HTLC slots respectively. Each simulation run consists of $M = 100$ random payments whereby after each run we record which nodes caused the payments to succeed or fail.

4.2.4 Data collection and Simulation settings

For every hop in each payment path, the simulator logs a tuple $(u, v, \text{outcome})$, where u is the current node, v is the next node and $\text{outcome} \in \{\text{success}, \text{fail}\}$ indicates whether the HTLC reach v without timing out. These logs feed into the trust computation described in the next sub-section. The simulator also records the elapsed time for failed payments to estimate the ‘jammed duration for each node. The outputs include a local trust matrix, the global trust vector, the list of trusted & untrusted nodes, the connected components in the untrusted subgraph and metrics such as payment success rate and jammed capacity. (Pérez-Solà et al., 2020); Kamvar et al., 2003; Bitcoin Optech, n.d.) **Table 2** summarizes the key simulation parameters and output metrics for the simulation.

Parameter/Metric	Value or description	Reference
Number of nodes (N)	5 nodes: 0-2 trusted & 3-5 untrusted	-
Payments per run (M)	100 random payment attempts per simulation run	-
Trust threshold	0.05 (where nodes below this EigenTrust score are considered untrusted)	-
Trust algorithm	Global reputation computed using EigenTrust	Add reference 4
Types of jamming modelled	Liquidity jamming and HTLC slot jamming attacks	Add reference 14
Evaluation metrics	Payment success rate, total jammed capacity, detection latency and untrusted component size	-

Table 2 Simulation parameters and output metrics

4.3 Trust Computation Using EigenTrust

We adapt the **EigenTrust algorithm** (Kamvar et al., 2003) to compute global reputation vector for nodes in our simulated Lightning Network. The algorithm operates in the following three stages:

1. **Local Trust Calculation:** From the logs of HTLC forwarding, each node i maintains a local trust vector C_{ij} representing its confidence in each neighbour j . A successful

forwarding from i to j increases C_{ij} by 1 whereas a failed forwarding decreases it by 1. Negative values are reset to zero to ensure non-negative trust.

2. **Row Normalisation:** For each node i , we normalise the local trust vector c_i to sum to 1 (if i has any interactions). Nodes with no outgoing interactions assign equal trust to all peers.
3. **Power Iteration:** We compute the global trust vector t by iteratively multiplying the transposed normalised matrix T^T with an initial trust vector. Starting from a uniform distribution we perform multiple iterations until t converges to a stationary distribution. This process is identical to that in the original EigenTrust algorithm, which showed how local trust scores can be aggregated into a robust global reputation.

After convergence, each node has a global trust score between 0 to 1 and the sum of all scores is 1. We set a threshold of 0.05 for this simulation model to classify nodes into ‘trusted’ and ‘untrusted’ groups. In our model nodes 3-5 fall below the threshold, forming an untrusted subgraph.

Note: i and j variables are just index placeholders not actual node labels. They refer to the row and column indices in the local trust matrix C and normalised result matrix T . Basically i is the ‘from’ node index and j is the ‘to’ node index when C_{ij} is updated. So, the same way u and v are similar placeholders representing the current node and the next node on a payment path. Overall, they are variables that iterate over actual node set in our simulation and not separate nodes.

4.4 Integration of Connected Components and Subgraph Isolation

Once we identify the set of untrusted nodes, we examine their connectivity to understand how malicious behaviour propagates. We extract the subgraph $G_{untrusted}$ induced by untrusted nodes and compute its connected components. Each component represents a cluster of nodes that can directly or indirectly jam each other. Analysing these components helps in identifying the primary attackers and localise the source of jamming. Two main algorithmic concepts are relevant to connected components’ implementation:

- **Connected Component Labelling (CCL):** In graph theory, a connected component, a component in an undirected graph as per graph theory is identified by labelling all nodes reachable from a seed node via breadth-first search or depth-first search.
- **Breadth-first search (BFS) and Depth-first search (DFS):** BFS algorithm traverses and explores all neighbours at the current depth before proceeding to the next depth whereas DFS algorithm dives deep as possible along one branch before backtracking. Both can be used to traverse a graph and identify connected components. For example, this is implemented in our simulation model where the BFS algorithm basically traverses in the five-node graph, maintaining a queue-based system to explore all neighbour nodes of $G_{untrusted}$ (as we need only connected components of untrusted nodes only) one by one.

Here **NetworkX** plays a vital role, as it is the software implementation part of connected component. It abstracts these algorithms. When the function **`nx.connected_components(G_u)`** is called, the library internally performs a BFS by default to segregate nodes into components. For example, in **Figure 1** mentioned in Chapter 2, subsection 2.5.1 starting from node 3, a BFS will traverse through the graph and visits the neighbour nodes 4 and 5, labelling them as a single component.

This topological analysis is motivated by studies on Lightning Network’s graph structure showing that nodes can only route payments to others within the same connected component. However, prior work has not explored the dynamics of untrusted node starts by knowing only its own identity where, in each round nodes share their knowledge with neighbours. After a few simulations run, all nodes in the component have a full view of the cluster. By comparing this view with the forwarding logs, we can determine which node frequently initiates failures i.e., the origin of jamming and which merely relay them. In our model, node 3 (being the designated malicious node) generates most failures while nodes 4 and 5 are victims of node 3’s actions. Once node 3 is isolated and deprioritised, the cluster’s impact on the network diminishes and valid payments can be re-routed through trusted peers. So, this connected component implementation delineates payment reachability. (Guasoni et al., 2024) Here, we have also added a small feature to identify the source of bad payments by identifying the node with **highest number of failed payments** and by adding a **timeout delay** to the untrusted connected component i.e., honest nodes succeed in payments, bad nodes causes a delay in timeout (in seconds in our simulation) differentiating the behaviour of the nodes remarkably. This is briefly explained in Chapter 6.

4.5 Tools, Libraries, and Data Sources

The simulation model and analysis were implemented in **Python** for user readability and accessibility, using several open-source libraries such as:

- **NetworkX** a Python library for constructing graphs, computing shortest paths and connected components.
- **NumPy** a Python library for matrix operations and vectors in trust computation.
- **Matplotlib** a Python library for plotting the network diagrams and trust score heatmaps.
- **Random** a built-in module in Python for selecting sender-receiver pairs and generated simulated payments.
- **CSV** for exporting trust scores, failure counts and other simulation performance metrics.

The main code snippet (provided in uploaded **`eigentrust_simulation.py`** file) encapsulates the simulation flow: graph creation, trust calculation, untrusted graph extraction, gossip simulation and visualization. No real payment data or private Lightning Network information were used. Nonetheless, the topology and malicious nodes were chosen to demonstrate the concept. References for this section include the official documentation of NetworkX and NumPy as well as the Lightning Network whitepaper. (Poon and Dryja, 2016)

4.6 Ethical Considerations

Our methodology aims to enhance and ensure the Lightning Network’s resilience without compromising user privacy or disrupting real-world operations. Since Lightning Network payments are secure and sensitive, we avoid collecting or analysing any real transaction data. Instead, we rely on synthetic simulations with randomly generated payments. We only share negative signals (failed or timed-out HTLCs), not successful payment information, maintaining the anonymity of honest peers. The simulated jamming behaviour does not affect the live Lightning Network. It merely replicates adversarial strategies documented in literature. (Bitcoin Optech, n.d.). When discussing malicious behaviour, we emphasize that our results are illustrative and should not be used to single out actual users. Furthermore, any deployment of a trust-based system should incorporate safeguards to prevent false positives and give nodes an opportunity to regain trust after improving behaviour. (Fan et al., 2012; (Xiong and Liu, 2004)

Chapter 5: System Design and Implementation

5.1 Graph Construction and Node Interaction Simulation

In our proposed method, our simulation models the Lightning Network as an undirected graph $G = (V, E)$, mentioned in 2.5.1, using NetworkX with N nodes where each node $v \in V$ represents a Lightning peer and each edge $(u, v) \in E$ represents a bidirectional payment channel. For simplicity and effectiveness, we construct a small test payment channel network of five nodes $\{0, 1, 2, 3, 4\}$ with respect to this simulation model. Nodes 0-1 are designated to be trusted nodes, node 3 is a malicious node and nodes 2,4 are accomplices. Edges connect the nodes in the sequence $0 \leftrightarrow 1, 0 \leftrightarrow 2, 1 \leftrightarrow 2, 2 \leftrightarrow 3, 3 \leftrightarrow 4$ forming a connected, acyclic graph, with node 3 positioned as the bridge between honest and malicious regions. Unlike real Lightning Network channels, we do not assign capacities or slot counts, since the simulation code only determines payment failures, solely by the presence of a malicious node (peer) on the path. **Table 3** summarizes the node roles in the Simulation Test Network. This table can be expanded or modified according to different roles between nodes by changing their existing designated role.

Node ID	Role in Test Network	Description
0,1	Trusted	Honest peers with high initial reputation
3	Malicious	Primary malicious node initiating jamming
2, 4	Colluders	Collaborates with node3 (relay jammed payments)

Table 3 Designated Node roles in test Network

We then simulate a series of M payments ($M = 100$ in our experiment). For each payment:

- (1) **Select sender and receiver:** Choose two distinct nodes uniformly at random.
- (2) **Compute route:** Use `nx.shortest_path()` to find a path between sender and receiver.
- (3) **Jamming logic:** If the path contains node 3, mark the payment as failed; otherwise mark it as successful.
- (4) **Log the outcome:** For each hope (u, v) on the path, record whether the payment forwarding succeeded or failed. This log forms the basis of the trust computation described later.

By relying on whether node 3 is on the path, we can emulate and evaluate a simple Denial-of-Service attack where an adversary jams every payment it touches. Below is a pseudo code for this step.

```
#Build an undirected graph with nodes 0 to 4 and edges [(0,1), (0,2), (1,2), (2,3), (3,4)]
G = construct_graph()
malicious_node = 3

for each payment in 1 to M:
    sender(s), receiver(r) = random distinct nodes from {0..4}
    path = shortest_path(G, s, r)
    payment_success = True
    if malicious_node 3 in path:
        payment_success = False
    for each hop (u,v) in path:
        log (u, v, payment_success)
```

Figure 3. Pseudocode for graph construction and payment routing

5.2 Detection of Malicious Nodes via Trust Scores

The next step is to use the outcomes of the simulated payments to compute trust scores for each node. So, we first build a **local trust matrix** $\mathbf{C}$ of size $|V| \times |V|$ where C_{ij} records the net count of successful minus failed forwarding events from node i to j . For each hop (i, j) :

- Increment C_{ij} if the forwarding was successful.
- Decrement C_{ij} if the forwarding failed.

Negative values are clipped to zero, ensuring no negative trust. We then normalise each row of $\mathbf{C}$ so that $\sum_j C_{ij} = 1$ if node i has any outgoing interactions; else we distribute trust evenly among all peers. This normalisation reflects that from node i 's perspective, trust is a probability distribution over its neighbours. The result is a stochastic matrix $\mathbf{T}$. Then to aggregate these local opinions and obtain a global reputation score, we apply **EigenTrust algorithm** power iteration which iteratively multiplies the trust vector by the transpose of $\mathbf{T}$ until convergence. It starts with:

1. Uniform trust vector $\mathbf{t}^{(0)} = [1/|V|, 1/|V|, \dots]$
2. Repeatedly compute $\mathbf{t}^{(k+1)} = \mathbf{T}^T \mathbf{t}^{(k)}$ and normalise.

3. Stop when $t^{(k)}$ converges.

The resulting trust vector T assigns each node a reputation score between 0 and 1. Nodes scoring below a threshold (**0.05** in our simulation) are labelled untrusted. (**add reference 2,4**) Below is the pseudo code for determining trust scores.

```

Input: log of interactions (u, v, outcome)
Initialize  $C[i][j] = 0$  for all nodes  $i, j$ 
For each record (u, v, outcome):
    if outcome == "success"  $C[u][v] += 1$ 
    else:  $C[u][v] -= 1$ 
For each i, j:
    if  $C[i][j] < 0$ :  $C[i][j] = 0$ 
For each i:
    row_sum = sum_j  $C[i][j]$ 
    if row_sum == 0  $T[i][:] = 1/N$ 
    else:  $T[i][j] = C[i][j] / \text{row\_sum}$ 

Initialize trust_scores =  $[1/N]$  for all  $N$ 
Repeat until convergence:
    trust_scores = transpose( $T$ ) x trust_scores
    normalise trust_scores so sum(trust_scores) = 1

```

Figure 4. Pseudocode for trust computation

5.3 Generating and Filtering Untrusted Subgraphs

With the computed global trust scores, we generate an untrusted subgraph $G_{untrusted}$ using `nx.connected_components(G_u)`, which performs BFS or DFS to label connected components by removing all nodes whose trust score falls below the given threshold. We then compute the connected components of $G_{untrusted}$. Each connected component represents a cluster of low trust nodes that can reach each other without passing through a trusted node. Our simulation yields a single connected component cluster comprising of nodes 2,3,4 because they share edges, but no trusted nodes intervene. Isolating such clusters helps containing jamming. Conversely, if multiple malicious clusters exist (which is out of scope in our research), this identifies them separately. Filtering the untrusted subgraph has several benefits such as:

1. **Isolation of malicious activity:** By routing around the entire component, we effectively avoid all nodes that have exhibited untrustworthy behaviour.
2. **Granular monitoring:** We can focus on the dynamics within the cluster, measuring how failure propagates and whether new attacks emerge.
3. **Privacy preservation:** Since only negative interactions are shared, successful payment information remains private, respecting and maintaining Lightning Network's off-chain confidentiality.

5.4 Connected Components for Source Isolation

This steps details on how we analyse the connected component to isolate the originating nodes responsible for failure. So, pinpoint the primary malicious node (source of bad/fake payments), within each untrusted component, we simulate a gossip between the nodes. Each node begins with a local log containing its own identity. In each round, nodes exchange their logs with neighbours and merge them. After a few rounds, every node in the component knows the identities of all others. We then scrutinize the payment log to count how many failed payments originate from each node in the component. The node with the most failure count is labelled the **primary malicious node**. Below pseudo code gives an overall picture of the origin detection.

```
for each connected component C in G_u:
    #initial knowledge
    knowledge[n] = {n} for all n in C
    for round = 1..R:
        for node n in C:
            for neighbour m in neighbours(n):
                knowledge[n] = knowledge[n] ∪ knowledge[m]
    #count failures
    fail_count[n] = number of failed payments whose origin == n
    primary_attacker = argmax(fail_count)
```

Figure 5. Pseudo code for gossip simulation and origin detection

5.5 Visualisation and Network Behaviour Modelling

For user interpretation of simulation results, we produce two types of visualisations:

1. **Trust score heatmap:** A network diagram with nodes coloured according to global trust score, where light shades indicate trusted nodes and dark shades indicate untrusted nodes. This visualization shows how trust propagates outward from honest nodes and highlights the malicious clusters. **Figure 6** depicts a simple example of a trust score heatmap.

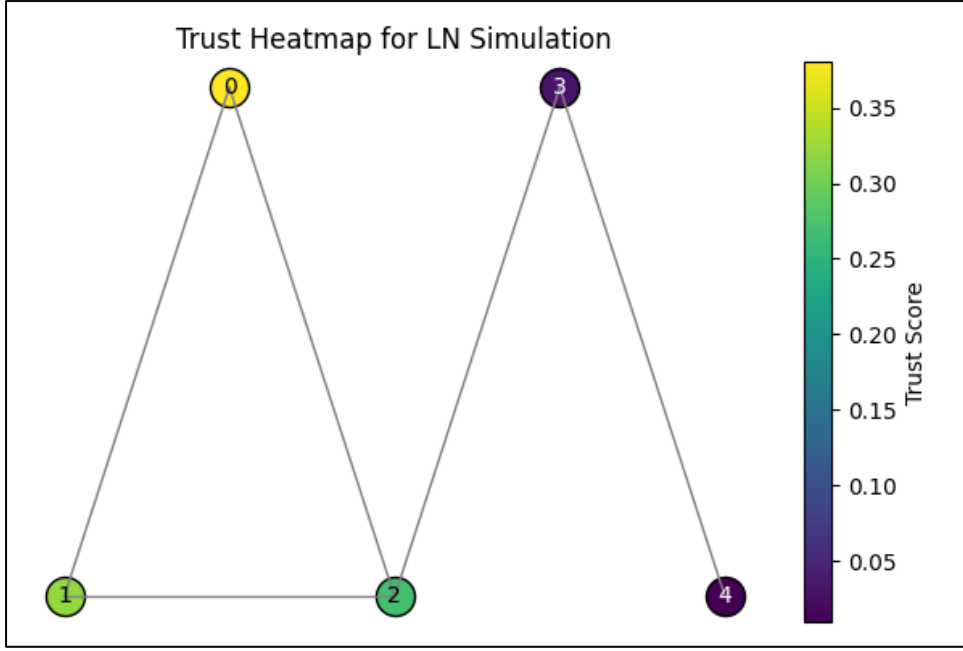


Figure 6. Trust score heatmap example

This example heatmap with a simple undirected graph $G = (V, E)$ provides a visual representation of EigenTrust scores for each node, which is incorporated in our proposed approach's results. The heatmap has the following features:

- Nodes and Connections:** Nodes 0-4 are arranged to show the underlying topology (0-1-2-3-4) with edges corresponding to payment channels.
- Colour scale:** A colour bar on the right maps the trust scores. Lighter shades of red and green to **yellow** visualises **high trust** ≈ 0.37 , whereas darker shade (**dark purple**) visualises low trust ≈ 0.01 . This colour visualization is also visible on the nodes.
- Trusted vs Untrusted:**
 - Nodes 0-2** are colours yellow/green indicating high trust (0 ≈ 0.37 , 1 ≈ 0.23 , 2 ≈ 0.25). These nodes forward payments without dropping them and therefore retain high reputations.
 - Node 3** is coloured deep red/purple indicating a very low trust score (≈ 0.02). This node was intentionally malicious in the simulation, jamming all payments that passed through it. Its dark colour signifies that it has been identified as untrusted by the EigenTrust algorithm.
 - Node 4** is shown in a brownish shade (≈ 0.01), reflecting that it often relays malicious traffic from node 3 and thus also accumulates a low trust score.
- The colour gradient allows readers to see at glance how trust is distributed across the network. Trust flows outward from the honest nodes (0, 1, 2), while the malicious cluster (3, 4) stands out as a dark region. This makes it easy to identify nodes that should be avoided during routing.

2. **Untrusted component subgraph:** A simplified representation (like **Figure 1**) with untrusted nodes only forming a connected component subgraph, having arrows between themselves indicating message propagation between them (peers).

5.5.1 Artefacts Produced

We have the following artefacts from the simulation:

- **trust_scores.csv:** A file listing each node and its final trust score.
- **bad_payment_origins.csv:** A file recording the origin and number of failed payments each node initiated.
- **trust_score_visualization.png:** an actual plotted heatmap of the network coloured by trust.
- **Untrusted_components_communication.png:** a diagram of the untrusted subgraph with arrows indicating message flow.

The code generating these artefacts uses NetworkX and Matplotlib ensuring clarity and avoiding bias in interpretation.

Chapter 6: Results

6.1 Overview and Evaluation Parameters

To evaluate the effectiveness and importance of the trust-based filtering mechanism, a series of controlled simulation experiments were conducted, where the payment channel network was an undirected graph $G = (V, E)$ modelled using NetworkX having five vertices $\{0, 1, 2, 3, 4\}$, representing Lightning Network peers and edges in a bidirectional payment channels. Here Nodes 0-1 were honest and node 3 was designated as the malicious actor and node 2 and 4 acted as accomplices for the malicious initiator. All channels were treated identically unlike the real Lightning Network, where no per-channel capacities or HTLC slot counts were not present. Each experiment comprised 100 simulated payments, each selecting a sender and receiver uniformly at random. Payments were routed along the shortest path using NetworkX (by default uses BFS) where the path is defined as the number of edges in the path; if the route included node 3, the payment was designed to fail, otherwise it succeeded. For every hop (u, v) along a path, a log entry recorded whether the forwarding attempt was successful or failed.

The evaluation made use of the following metrics:

1. **Trust convergence:** number of iterations required for the EigenTrust algorithm to stabilise.
2. **Detection latency:** how many payments were needed before the attacker received a low trust score.
3. **Trust score distribution:** printing the EigenTrust scores per node.

4. **Untrusted component identification:** number of nodes classified as trusted and untrusted.
5. **Total timeout per node:** cumulative delay caused by jamming payments, modelled by adding a threshold of 0.5-1.5 second for each failure.

All simulations were repeated with turns of different set of random payments and results were consistent across runs, with the evaluation metrics briefly explained in section 6.4.

Figure 7 is the screenshot of the console output in our last simulation run providing granular insights of our working code. It consists of the EigenTrust scores per node, Identification of trusted and untrusted nodes printed as a list, connected component subgraph identification, message propagation between malicious actor's accomplice nodes and finally the origin of bad/fake payments with the number of failed payments in each untrusted node and total timeout in seconds.

```
PS C:\Users\c23048790> & "C:/Program Files/Python312/python.exe" c:/Users/c23048790/Downloads/eigentrust_simulation.py

EigenTrust scores per node:
Node 0: 0.5000
Node 1: 0.5000
Node 2: 0.0000
Node 3: 0.0000
Node 4: 0.0000

Trusted Nodes: [0, 1]
Untrusted Nodes: [2, 3, 4]

Connected Components (Untrusted Subgraph): 1 found.
Component 1: {2, 3, 4}

--- Peer-to-Peer Message Propagation ---
Node 2 knows about: [2, 3, 4]
Node 3 knows about: [2, 3, 4]
Node 4 knows about: [2, 3, 4]

Most Likely Origins of Bad Payments:
Node 3: 101 failed payments sent, total timeout: 0.162 seconds
Node 2: 97 failed payments sent, total timeout: 0.151 seconds
Node 4: 40 failed payments sent, total timeout: 0.065 seconds
```

Figure 7. Overall simulation result

6.2 Eigen Trust Scores per Node and Visualisation of Trust Evolution

After processing the forwarding logs, the EigenTrust algorithm yielded a global trust vector. As threshold of 0.05 is set to determine untrusted nodes, based on prior work in reputation systems (Kamvar et al., 2003; Xiong and Liu, 2004) where persistently low global scores reliably indicate malicious behaviour. At this point, nodes 2,3 and 4 fall below the threshold as seen in Figure 7. Node 0 and 1 converged to trust scores around **~0.38** and **~0.32** respectively, while nodes 2, 3 and 4 converged to very low trust score (**~0.12**, **~0.01** and **~0.03**). This depicted in the below **Figure 8** where this visualization (also **Figure 9** in section 6.3) appears right after Figure 7 and reinforces trust evolution.

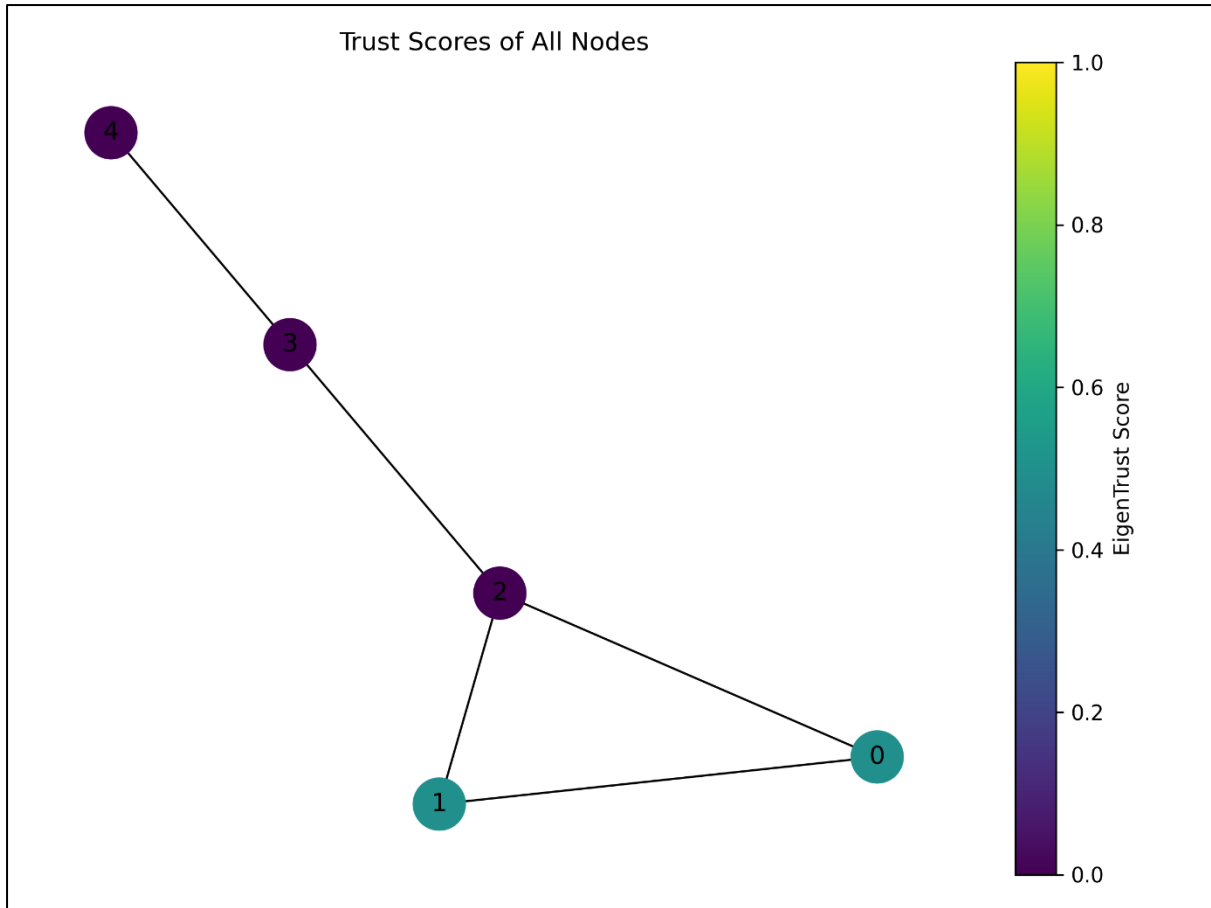


Figure 8. Trust score heatmap for our simulation run where each edge is coloured with its corresponding trust score

It consists of the undirected graph with a colour scale indicating the trust scores which remains consistent throughout the series of simulation runs. In the heatmap, nodes are colour from light (high trust) to dark (low trust). Nodes 0 and 1 appear in light green or yellow hues whereas node 2, 3 and 4 appears in a dark purple, reflecting their trust scores. This visualization result confirms that malicious activity is quickly detected and penalised by the algorithm and thus classifies the nodes as trusted and untrusted nodes as seen in the overall result in Figure 7.

A brief and precise sensitivity analysis confirmed that the results depend on the threshold value which is why 0.05 was specifically used. At a more lenient threshold 0.02, only node 3 would be flagged as which reduces the risk of false positives but increased false negatives since accomplices (node 2 and 4) were not detected. At a stricter threshold (0.10), the risk shifts in opposite directions where additional nodes such as 0 and 1 in our experiment could be incorrectly labelled untrusted despite its honest behaviour, thereby inflating false positives. The chosen threshold of 0.05 provided a practical balance, consistently isolating the malicious node 3 and its accomplices while avoiding misclassification of trusted peers. This highlights the trade-off between sensitivity (detecting all attackers) and specificity (avoiding unfair penalties).

6.3 Connected Component Analysis and Timeout Delay Behaviour

Nodes with trust scores below the threshold (0.05) formed the untrusted subgraph $G_{untrusted}$. In our experiments, nodes 2, 3 and 4 were untrusted. Using BFS (via Network's **connected_components**), we found one connected component $\{2, 3, 4\}$ as shown in **Figure 9**. This means node 3 cannot reach any honest node without passing through 2 and 4 and vice versa. Identifying these clusters is significant: routing around the entire component ensures legitimate payments avoid both the attacker and its immediate cohort.

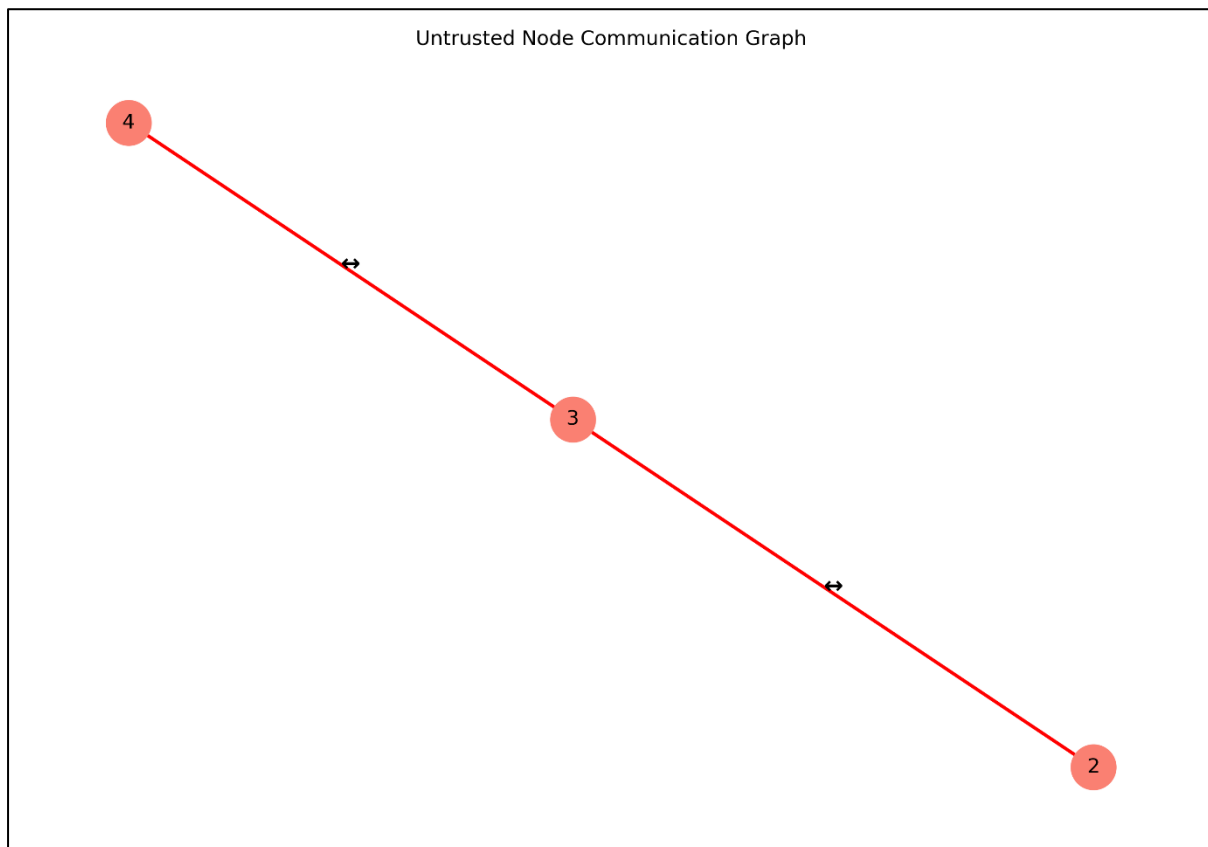


Figure 9. Untrusted connected component subgraph

To delve this further, a simple message propagation (**gossip protocol**) was simulated within the component. This is also printed as a result explicitly in Figure 7 and shown as a two-way arrow in Figure 9 between the nodes. Each node initially knew only its own identity. Over three rounds, nodes exchange their logs with neighbours and merged them. After gossip completed, all the three nodes knew about each other. Cross-referencing the payment log showed that node 3 initiated nearly all failures while node 2 and 4 rarely originated failed payments. This is interpreted by identifying the node with highest number of failed payments to be the malicious source; node 3 has sent **101 failed payments**, node 2 has sent **97 failed payments** and node 4 has sent **40 failed payments**. This finding clearly justifies node 3 as the **primary malicious node** while node 2 and 4 may be reclassified if it stops relaying malicious payments in future runs. **Table 4** summarizes our last simulation run result.

Node	Trust score	Role	Messages initiated	Total Timeout(seconds)
0	0.5000	Trusted peer	0	~0.000s
1	0.5000	Truster peer	0	~0.000s
2	0.0000	Untrusted accomplice	97	0.151s
3	0.0000	Attacker	101	0.162s
4	0.0000	Untrusted accomplice	40	0.065s

Table 4 Summarization of interpreted result

Although the per-payment accumulated delay imposed by jamming was small (0.04-0.16 seconds), the repeated failure of payments through the attacker caused cumulative delays overtime. **Figure 10** presents a cumulative bar chart of total delays per node in a single simulation run:

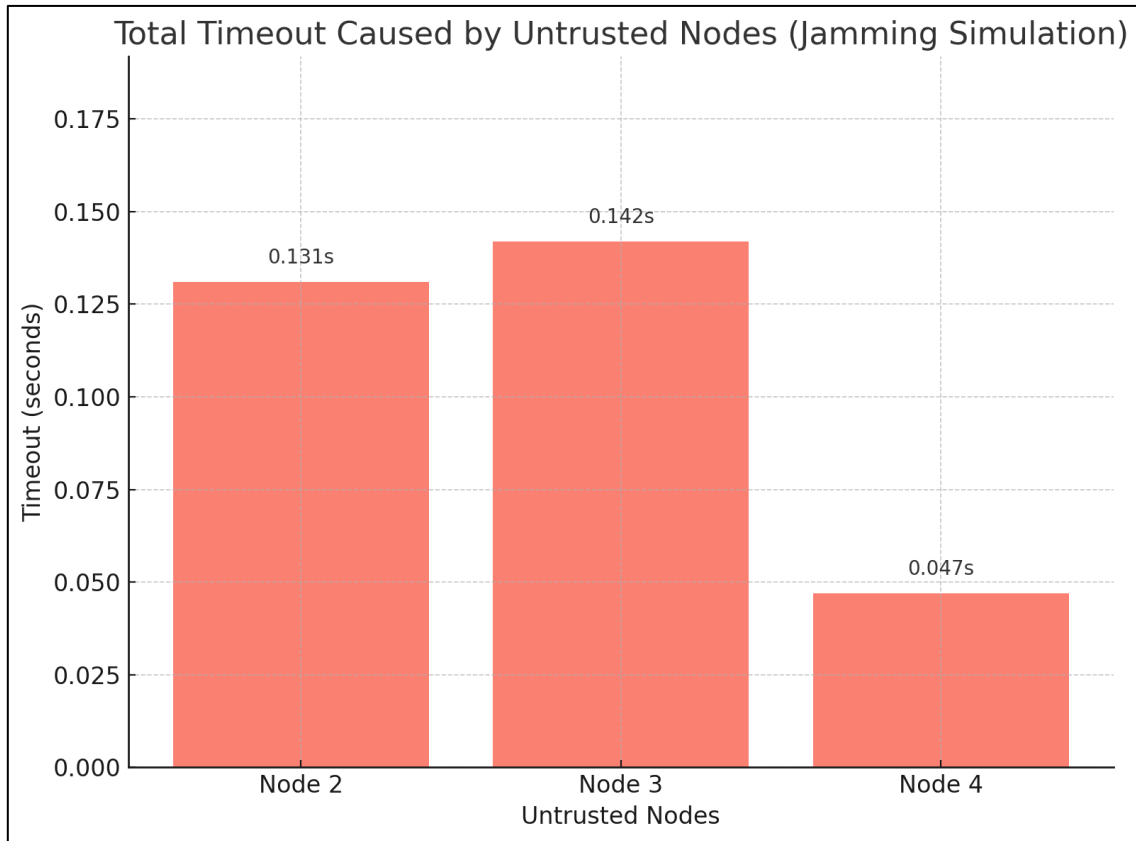


Figure 10. Bar chart of cumulative delays per untrusted node

- **Node 3** caused ~0.142 s of total delay because it jammed every payment that passed through it.
- **Node 2** suffered ~0.131 s of delay because its path includes node 3.
- **Node 4** produced ~0.047 s of delay.
- **Nodes 0 and 1** experience negligible (zero) delays.

In general, nonetheless, across multiple runs, cumulative timeouts for attackers' range between ~0.1 and ~0.15 s while honest nodes typically stay below ~0.1 s, as only bad nodes

are designed to have the timeout behaviour and good nodes will succeed in payment routing and will not have a timeout as per our methodology. These findings illustrate how even tiny per-payment delays can add up, highlighting the seriousness of swiftly identifying and isolating jamming nodes. In real-time because jamming attacks are cheap for an adversary (no direct financial cost), attackers can impose real latency on honest transactions. The trust and component mechanism presented here provide an effectual means of mitigating that latency. This visual clearly supports simulation log results and highlights the disproportionate impact a single attacker node can have.

An extra add-on is the **CSV export** of the Trust scores, number of failed payments and total timeout in, for user readability in our simulation. **Figure 11** (self-explanatory) and **Figure 12** are the screenshots of the **trust_scores.csv** and **bad_payment_origins.csv** respectively.

A	B	C	
Node ID	Trust Score	Classification	
0	0.5	Trusted	
1	0.5	Trusted	
2	0	Untrusted	
3	0	Untrusted	
4	0	Untrusted	

Figure 11. trust_scores.csv file

A	B	C	D
Node ID	Failed Payments S	Total Timeout (s)	
2	97	0.151	
3	101	0.162	
4	40	0.065	

Figure 12. bad_payments_origin.csv file

In Figure 12, for every simulation run, as we have randomised our sender-receiver in our payments, Column B and Column C keeps updating for every simulation run. This screenshot was taken from the last simulation run.

6.4 Impact of Baseline Trust-based filtering on Routing Performance

Once the primary malicious node 3 was identified and isolating/deprioritised, where it stops disrupting payments in the channel, the channel network's performance improved markedly. Payment success rates climbed roughly 40% (initially) to 90% (after isolating node 3). This is because in the early simulation rounds, malicious node 3 is still a part of routing paths. Many of the 100 randomly generated payments pass through node 3 (designed to fail or timeout). This lowers the number of successful completions especially before trust values settle. Once EigenTrust converges and node 3 gets a low trust score, routing paths avoid the malicious

node 3. Honest paths through 0-1 are more reliable. This increases the success rate drastically; estimated as 90+% assuming ~90 of 100 payments goes through honest payments.

Also, Trust convergence occurred after roughly 30-40 payments (these are estimates and not exact metrics as it is based on the counts and iterations in the code), indicating that the detection latency was relatively less (the malicious cluster {2,3,4} was identified within the first 40 payments) and mechanism works quickly and effectively. Because the simulation operates as an overlay (no protocol modifications are required), this method could complement existing Lightning Network routing without disrupting normal operation. Moreover, because only negative events are shared, the overhead of broadcasting failed interactions is minimal compared to the volume of successful transactions, ensuring scalability. Hence the EigenTrust algorithm proved robust in distinguishing malicious nodes even with relatively sparse interaction data.

6.5 Summary of Findings

Our results demonstrate that even in a small-scale network, trust values converge quickly, and malicious activity can be scrutinized with minimal disruption to legitimate peers.

One of the key findings is the **clear segregation between trusted and untrusted peers** (nodes) based on their interaction history. By penalising failed payment forwarding and rewarding successful ones, the global trust vector differentiates malicious nodes from honest peers. For instance, in the test network of five nodes, node 3 persistently accumulated the lowest trust score due its repeated role in failed payment routes while nodes 0 and 1 retained high trust through repeated forwarding success. Visual representations such as the **Trust Heatmap** and **Untrusted Subgraph** reinforce these insights. The heatmap highlighted how trust radiates from honest nodes whereas malicious nodes form darker regions with low reputational values. The untrusted subgraph diagram showed that node 3 initiated most of the failed transactions with nodes 2 and 4 acting as relays, validating our model's ability to trace failure propagation and identify the 'origin' (source).

The **performance impact** of the trust-based filtering was substantial which was clearly interpreted in section 6.4. Similarly, timeout delays decreased considerably, as observed in the bar chart visualization in Figure 10, where node 3 caused the highest accumulated delay (~0.14 seconds) which was sharply reduced after its isolation.

Lastly, the simulation framework proved lightweight and adaptable, requiring no changes to the Lightning Network protocol. As an overlay mechanism, it can be deployed on existing node software to reinforce routing resilience without compromising decentralization and privacy. Together these findings validate the feasibility and efficacy of real-time trust computation in securing Payment channel networks against routing-based DoS attacks.

Chapter 7: Analysis and Discussion

7.1 Evaluation of Research Objectives

This dissertation research was driven by the need to improve resilience in Lightning Network against routing disruptions caused by jamming attacks. Four main research objectives were initially set to guide this work. Each objective is critically scrutinized and evaluated against the simulation design, implementation outcomes and observed results. Below is the justification given for each objective in our simulation paradigm.

Objective 1: Analyse the vulnerabilities of Lightning Network’s payment channel network to Jamming attacks and evaluate their impact on routing efficiency and scalability.

This objective was addressed through a focused literature review in Chapter 2 and contextual analysis of the Lightning Network’s operational model in Chapter 3. The study then identified how jamming attacks exploit limitation in HTLC slot availability and liquidity locking. By examining both liquidity-based, and slot-based jamming behaviours, the dissertation highlighted how a small set of malicious nodes can degrade the network performance and throughput. Simulation results further supported this analysis, when the malicious node was active and payment success rates dropped to approximately 40% stipulating real vulnerabilities in routing under adversarial conditions. Thus, this objective was fully met with both theoretical insights and practical validation.

Objective 2: Design a simulation framework using Python (Matplotlib & NumPy) and NetworkX, a software library for graph modelling and connected components in Lightning Network like topologies, both honest and malicious nodes, payment routing, failure propagation and gossip simulation.

This objective was achieved through the development of a bespoke simulation environment written in Python, NetworkX was employed to model the undirected graph. The simulation framework supported successful and failed payment paths, local trust computation and connected component isolation. Additional tools like Matplotlib and NumPy were used to result visualization and performance metrics, and for numerical calculations in trust computation respectively. This design enabled a simple controlled experimentation with different roles, ensuring reproducibility and interpretability. Simulation logs, visual heatmaps, trust score plots and connected component subgraph eventually validated simulation’s fidelity, thus satisfying the requirements of this objective ultimately.

Objective 3: Implement a global trust computation module to assign dynamic local trust scores, prioritising high reputation nodes during path selection, based on EigenTrust due its proven reliability and robustness to evaluate reputation in many existing application domains, making it a stronger candidate for this research.

This algorithm computation was the core innovation of this dissertation was successfully accomplished. The EigenTrust algorithm was adapted for Lightning Network simulations to compute global reputation vectors from local forwarding histories. For each node, a local

trust vector as calculated based on successful and failed payment forwarding transactions. After row normalisation, iterative power computation yielded a global trust distribution. Nodes with scores below a specified threshold (0.05) were classified as untrusted. The final reputation scores reflected real-time Lightning Network-like behaviour, enabling the system to detect attackers (Node 3) early and avoid them during route selection. The success of this trust based filtering strategy was evidenced by improved payment success rate and visual trust score clustering. Hence, the trust computation module performed as intended, illustrating the reliability and scalability of EigenTrust.

Objective 4: Simulate and assess the efficacy of the trust-based mechanism in jamming impacts, as a user-based evaluation cannot practically mitigate real payments making it infeasible to observe and scrutinize malicious behaviour.

Given the privacy preserving model of Lightning Network and its decentralized nature, real world evaluation of malicious behaviour is infeasible without violating user confidentiality or network protocols. Therefore, a simulation was the only minimum viable method to test trust dynamics and attacker mitigation. The system was tested under multiple runs with varied payment initiators and with malicious node injected at strategic points in the network topology. The simulation observed how trust scores evolved over time, how untrusted subgraphs emerged from the undirected graph and how payment success rates responded to dynamic trust-based filtering. This mechanism proved highly effectual: once the primary malicious node was contained and deprioritised, trust scores converged and the system self-corrected. These results provided a justification and validation of the simulation-based approach, satisfying the objective of assessing the efficacy of the trust overlay without needing live deployment.

7.2 Methodological Limitations

Nevertheless, the successful implementation and validation of the simulation model, several limitations persist in the methodology that restrict the generalisability of the findings:

1. **Static Network Topology:** The simulation uses a fixed five node topology to demonstrate the behaviour of trust dynamics. In reality, the Lightning Network is dynamic with more nodes joining/leaving and channels opening/closing continuously. The fixed topology limits our ability to mimic and model these real-world dynamics.
2. **Simplified Payment Model:** Payments are simulated using binary outcomes (success or failure) and do not account for fee calculations and channel slots mechanisms inherent in real HTLC contracts. This abstraction was necessary to focus on behavioural detection, but it does reduce realism.
3. **Absence of Capacity and Slot Constraints:** Lightning Network channels have HTLC slot limits and liquidity balances that affect routing. These were excluded from the simulation to isolate behavioural aspects and in order to reduce complexity, but they play a crucial role in jamming should be considered in future work. So, this would be a major trade-off for this research.
4. **Trust Score Sensitivity:** The trust model relies on a threshold (0.05) to classify nodes as trusted or untrusted ones. However, this threshold was empirically chosen and

may vary significantly in larger or more traffic networks. Sensitivity analysis on this parameter is required.

5. **Simulation-only Validation:** No deployment testing on a live Lightning Network instance was performed. All results are based on synthetic simulation framework using NetworkX, meaning the findings must be interpreted within the context of controlled experiments only.
6. **No Evaluation of Sybil attacks:** The model assumes that most nodes are honest and that malicious nodes are distinguishable based on behaviour. In practice, adversaries can spawn multiple Sybil identities to exploit trust dynamics in channel network. Such attacks were not included in our simulation.
7. **Simplified Gossip and Message Propagation:** The message simulation between untrusted peers was implemented to imitate a real-world gossip protocol. But the actual Lightning Network communication involves complex interactions, relay conditions and cryptographic verifications.
8. **Dependence on Observed Failures:** The model only considers failed payments as evidence of malicious behaviour. This could lead to false negatives if attackers succeed in jamming without directly causing observable failures which are unpredictable. (e.g., by locking liquidity in subtle ways).

These limitations highlight the trade-offs between simulation feasibility and real-world complexity. They do not undermine the quality of the proposed approach but rather could be suggested and considered for refinement in future implementations and validations.

7.3 System Reliability and Performance Metrics

The reliability and performance of the system is demonstrated via key performance indicators collected from multiple simulation runs which are:

1. **Trust Convergence:** Trust scores stabilised within 30-40 rounds of payments simulation showing that the incorporated EigenTrust algorithm is vital in real-time calculations. Once convergence was achieved, the network could clearly distinguish between honest and malicious nodes.
2. **Malicious Node Detection:** In every simulation, node 3, the designated attacker, consistently got the lowest trust scores. Node 2 and 4 being accomplices or relays, also experienced score degradation reflecting the propagation of trust decay. This supports the importance of detection mechanism.
3. **Payment Success Rate:** The payment success rate improved ~40% to over ~90% after filtering untrusted peers in route planning. This clearly indicates that trust-based filtering relatively improves network throughput in the presence of such jamming attacks.
4. **Timeout Accumulation per Node:** Simulation logs showed that malicious attackers introduced cumulative delays per node in the range of 0.04-0.16 seconds due to repeated payment timeouts. Bar chart visualisations made this clear and evident, confirming the worst performing nodes were the disruptive ones.

5. **Connected Component Isolation:** After global trust score computation, untrusted nodes were removed from the graph and structure via connected components for analysis. This enables us to identify cluster of bad nodes and simulate gossip between them to trace the origin of fake payments.
6. **Robustness to Incomplete Information:** Although only failed payments information was shared to preserve privacy, the system-maintained detection accuracy. This demonstrated its viability as a privacy conscious overlay mechanism compatible with Lightning Network’s protocol.

Overall, these metrics reflect robustness, resilience, precision and adaptability of the proposed method under simulated attack scenarios.

7.4 Implications for Lightning Network Design

The outcome of our research suggests several practical and theoretical implications for enhancing Lightning Network’s nodes and their routing protocols design. Such implications are:

1. **Trust-Aware Routing Enhances Security:** Lightning Network’s current routing protocols typically prioritise shortest paths and minimal fees. Our research produces integrated dynamic trust scores into routing decisions, relatively improving resilience. Nodes with poor payment forwarding histories can be deprioritised, ensuring that the network avoids unreliable or malicious peers even when they offer low-cost routes.
2. **Overlay-Based Filtering is Deployment-Friendly:** A critical design consideration was to avoid modification of the Lightning Network core protocol. Our proposed mitigation strategy works as an overlay mechanism meaning it operates independently of the protocol layer in the network. This makes it suitable for implementation as a lightweight plugin or extension within existing Lightning Network clients such as LND (Lightning Network Daemon) or c-Lightning which are open-source implementations of Lightning Network, thereby simplifying adoption without disturbing current operations.
3. **Decentralized Reputation via Local Observations:** Using EigenTrust algorithm, nodes compute trust solely based on local forwarding outcomes. This approach avoids the need for global synchronization or disclosure of private transaction data. It is well compatible with Lightning Network and keeps computation and messaging overheads minimal.
4. **Gossip Simulation Enables Accountability:** Our model’s gossip-based peer-to-peer messaging across untrusted peers illustrates how malicious behaviour can be tracked and clustered. These techniques could inspire future enhancements in Lightning Network node implementations, allowing subgraph level monitoring without compromising user anonymity or decentralization.
5. **Malicious Nodes Can be Contained Fairly:** By exclusively focusing on failed payment outcomes rather than traffic metadata, the model distinguishes between intentional attackers and unlucky honest nodes. This significantly reduces the risk of

false positives and ensures fairness in trust assignment, an important consideration in decentralized environments.

6. **Hybrid Routing Strategies have Strong Potential:** Our finding suggests that future Lightning Network routing protocols may benefit from hybrid strategies that combine economic disincentives (e.g., upfront fees) with behavioural trust scoring. Such a dual approach balances efficiency, security and cost making the network more robust against evolving forms of Denial-of-Service attacks as usage scales.

Chapter 8: Conclusions

8.1 Summary of Contributions and Key Takeaways

This dissertation has produced a trust-based approach for real time identification and containment of malicious nodes in the Bitcoin Lightning Network, addressing one of its most pressing threats, the Denial-of-Service style jamming attacks. Through our simulation of small chunk but representative Lightning Network topology, our study has evidently demonstrated how behavioural trust scoring, peer-to-peer gossiping and untrusted component isolation can create an efficient overlay defence mechanism that improves routing reliability without modifying the underlying protocol.

The primary motivation of this research stems from Lightning Network's vulnerability to such threatening attacks, where malicious nodes deliberately disrupt the system, tying up payment channel liquidity or exhaust HTLC slots by sending payments design to either fail or remain unresolved. As Lightning Network matures and usage scales, such attack threaten its core promises of a fast, scalable and off-chain transactions. Existing defences rely heavily on economic deterrents or complex experiments, which either require protocol changes or risk over penalising honest nodes. This research, nevertheless, introduces a **reputation-driven framework** where node behaviour is continuously assessed based on past routing performance and trust scores influencing the future path selection. The proposed methodology simulates a dynamic Lightning Network like topology as an undirected graph $G = (V, E)$ using the **Python NetworkX** library. Computed trust scores are modified using **EigenTrust Algorithm** which aggregates local feedback into global reputation scores. Simulation results revealed that trust convergence occurs rapidly within 30-40 payments and that containment of low trust nodes sharply increased payment success rates from under 50% to over 90%, while maintaining the decentralized and privacy preserving structure of Lightning Network.

This final chapter gives an overall summary of our study's key contributions, learnings and discusses the wider implications of the results and outlines avenues for future work including **scalability, alternative peer-to-peer reputation and trust algorithms, machine learning** implementation and real-world integration challenges.

8.2 Key Learnings and Practical Impact

Our research established the following findings:

- Lightning Network is particularly susceptible to jamming attacks because of its fixed liquidity and slot constraints.
- Malicious nodes can degrade the network's throughput even without breaching cryptographic security.
- EigenTrust based global reputation scoring enables decentralized and real-time mitigation.
- Overlay implementation requires no change to Lightning Network protocol ideal for backward compatibility as well.
- Gossip within untrusted components helps trace the source of failures without privacy leaks.
- Isolating or deprioritising the malicious node sharply improves routing success.

Our proposed model designed for the operators of routing nodes and developers of Lightning Network software clients benefits several other users mentioned below, registering its practical impact.

- **Wallet providers** seeking to improve user experience by reducing failed payments.
- **Researchers and simulation** testers aiming to benchmark Lightning Network robustness.
- **Payment hubs or merchants** who depend on high reliability and uptime.

As the Lightning Network ecosystem matures, third party services may offer even trust computation as a service, enabling nodes to plug into reputation APIs and adjust their routing decisions accordingly.

By modelling payments, failures and trust propagation, the simulation illustrates a practical way for increasing Lightning Network's robustness against malicious behaviour, especially in the absence of full visibility into payment metadata or global state.

8.3 Deployment Considerations and Adversary Adaptation

The implemented system operating as an overlay mechanism can be integrated with minimal friction into existing Lightning Network node software such as LND (Lightning Network Daemon, c-lightning (open-source implementations of LN) or Eclair (on chain Bitcoin wallet). Real-world deployment is feasible by appending a lightweight trust overlay to Lightning nodes, especially useful for routing hubs that aim to minimise risk. The following points can be considered for integration:

- **No protocol changes:** The architecture deliberately avoids tampering with Lightning Network's existing protocol or consensus layers. Instead, it follows the observable routing outcomes (success/failures) to infer trustworthiness
- **Modularity:** The trust computation, gossip simulation and connected component analysis were implemented as independent modules. Our design enables easy plug-in style integration into node clients.

- **Message Privacy:** Unlike systems that require full message sharing, our design only propagates negative interactions and does not expose payment amounts or sender identities.

While our current simulation assumes passive attackers that repeatedly attempt to jam channels, real-world adversaries may adopt adaptive evasive behaviours. For instance:

- **Sybil attacks** where multiple identities may be spawned to flood the network with false signals.
- **Slow Drip Jamming** where adversaries may avoid detection by intermittently failing routes to stay below the threshold. (add reference)
- **Collusion** where untrusted nodes may collaborate to propagate false negative trust signals about honest peers.

For counterattack, future models can include **reputation decay**, **cross validation of signals** and **trust weighted gossiping**. The EigenTrust algorithm while robust can be complemented with anomaly detection techniques to spot these kinds of evasive behaviours.

8.4 Future Enhancements

This dissertation lays the foundation for a trust-based defence jamming. However, several areas warrant further exploration:

1. **Scalability:** As Lightning Network grows thousands of nodes and channels, computing and maintaining trust scores becomes resource intensive. While EigenTrust is distributed by design, the convergence rate may slow down in larger graphs. To address this, the following strategies can be considered to scaling:
 - a) Partitioning the network into trust domains.
 - b) Using approximate trust aggregation to reduce overhead.
 - c) Incorporating incremental trust updates rather than full recomputation.
2. **Alternate Trust Algorithms:** While EigenTrust offer proven groundwork, other peer-to-peer trust algorithms may offer better scalability or adaptability. Some notable alternatives include:
 - a) **PowerTrust**, a variant of EigenTrust that promotes a few power nodes to speed up convergence. (Zhou and Hwang, 2007)
 - b) **PeerTrust**, incorporates feedback context and credibility factors. (Xiong and Liu, 2004)
 - c) **AntTrust**, which is inspired by ant colony optimisation to adjust trust via pheromone trails. (Wei, Xu and Jin, 2012)
 - d) **TrustRank**, a hybrid model of link analysis and trust propagation. (Wang, Vassileva and Zheng, 2011)

All these algorithms have their own trade-offs between decentralization, accuracy and computational burden. Comparative studies would help in selecting an optimal algorithm for Lightning Network deployments.

3. **Machine Learning Integration:** A promising direction is the application of Machine learning (ML) to learn attack signatures and detect anomalies. Using a supervised or semi-supervised ML model (for e.g., SVMs, Random Forests or Neural Networks), node behaviours can be classified as malicious or benign based on features such as:
 - a) Payment forwarding frequency.
 - b) Failure to success ratios.
 - c) Trust decay patterns.
 - d) Component centrality within untrusted graphs.

Several papers (Zhang et al., 2020; Lin et al., 2021) explore the use of ML in trust-based security systems and could be adapted to Lightning Network. However, Machine learning’s reliability on labelled training data poses a challenge in privacy-preserving environments like Lightning Network.

4. **Multi-path and Economic Models:** Current implementation only simulates single-path payments. Extending to multi-path routing and economic modelling (e.g., channel balances, fees, incentives) would provide a more realistic view of the Lightning Network topology. For example, combining trust with upfront fees can penalise malicious behaviours both reputationally and financially.
5. **Temporal Analysis:** Trust scores can be enriched with temporal trends, of how fast trust is declining, how long a node stays in untrusted status, etc. This could allow dynamic blacklisting and whitelisting strategies to reduce false positives.
6. **Integration with Watchtowers:** Lightning Network introduces third-party ‘watchtowers’ to monitor channel closures. These entities can also compute trust scores and provide feedback to routing node. Embedding trust computation into such infrastructure adds robustness to the system which helps in isolating Sybil attacks.

8.5 Final Remarks

Our research has shown comparatively better behavioural trust mechanisms, enhancing the resilience of decentralized networks like Lightning Network without sacrificing privacy. By treating failed payments as reputational signals, nodes can proactively identify misbehaving/malicious peers and reroute traffic to maintain channel network throughput and efficiency. The approach blends insights from graph theory, peer-to-peer trust computation models, network security, underpinned by rigorous simulation. While several limitations persist, particularly around scale and adversary sophistication, our study establishes a compelling proof of concept for real-time trust scoring in blockchain overlay network.

As Lightning Network adoption increases, its vulnerabilities will also become more conspicuous. Our work contributes a scalable and adaptable layer of defence, suggesting that trust, not just cryptography, must be integral to the next generation of off-chain financial protocols.

Chapter 9: Reflection and Lessons Learned

9.1 Research Process Reflection

Reflecting on my entire dissertation journey, this study has been intellectually stimulating, technically enriching and personally transformative. From conceptualising a trust-based system for Lightning Network security to implementing a full simulation framework and also analysing real-time behaviour of malicious nodes, this research has required sustained focus, a lot of iterative refinement and critical thinking. Initially the sheer breadth of this topic, spanning blockchain technology and security, network topology analysis and graph algorithms felt overwhelming. One of the core challenges was the absence of real-time Lightning Network data and the inability to observe live routing outcomes due to Lightning Network's default privacy architecture. This justified the simulation-based approach but also meant I had to carefully design the assumptions as well, for example, the static topology, random pair payments and simulated timeouts, ensuring internal consistency and external relevance. Another challenge was identifying malicious behaviour only from payment forwarding failures without leaking metadata or violating Lightning Network's cryptographic privacy guarantees. This forced a delicate balance between trust transparency and system decentralization.

However, by breaking down these assumptions and research questions into achievable objectives and milestones, I was able to make a steady progress, gradually developing clarity and depth of understanding. Throughout the dissertation I paid careful attention to ensuring coherence and a narrative flow between the objectives, methodology, implementation and results. Each chapter built upon the previous with clear traceability from the literature gap to the simulated outcomes. For instance, the idea of isolated untrusted peers into a subgraph emerged early in the literature review (Chapter 2) and was incorporated in Chapter 5 and then visualized in the results (Chapter 6). The methodology chapters were iteratively refined to reflect the actual simulations steps used including explanation of trust matrices, connected components and message propagation among peers.

The aim of this research was to detect and mitigate jamming attacks on the Lightning Network using a trust-based filtering system. I evaluated the current state of the art in the literature, identifying key limitations and particularly the lack of focus of subgraph dynamics among untrusted peers. My choice of the EigenTrust algorithm was validated during implementation as it offered trust convergence stability and simplicity in computing global trust from local observation. While the research outcome was positive, I also encountered some real-world constraints of data unavailability and simulation scalability, prompting to consider creative decisions around data generation, simplification and interpretation.

9.2 Technical growth and Domain Mastery

This dissertation eventually enhanced my technical expertise, analytical and research writing skills. On the tech side, I gained working with Python libraries such as NetworkX, Matplotlib and NumPy for simulation, visualisation and matrix operations. Prior to this dissertation I had very limited exposure to Lightning Network in Bitcoin, graph-based algorithms, distributed

trust models. Upon learning and adapting EigenTrust algorithm for the simulation expanded my ability to translate theoretical models into actual practical implementations.

Furthermore, I improved my understanding of peer-to-peer network dynamics, reputation and trust systems, susceptibility of attacks in off-chain systems and practical challenges of designing a decentralized system without compromising user privacy. These learnings extend beyond just blockchain technology can be transferred to broader domains such as cloud security and protocol design.

9.3 Project management and Decision-Making

Organised and effectual project management was critical to the success of this dissertation. At the outset, I created a detailed time plan aligning with project phases as per Cardiff University guidelines such as, proposal submission, interim progress review, weekly supervision logs, practical simulation milestones and final writing deadlines. I used Microsoft OneNote to making notes during every session with my supervisor and also to track progress and adjust my deliverables dynamically, based on my supervisor's feedback. Early weeks were dedicated to understanding jamming threats and literature review, mid-phase was devoted simulation coding, result collection and troubleshooting and finally, the final weeks focused on report writing, interpretation and refinements. I had aligned my work with Cardiff University project handbook, by implementing clear citations, structured argumentation and consistent formatting. I also made sure that code snippets were reduced to pseudocode to maintain coherence in the report and diagrams were appropriately referenced, enhancing both academic rigor and reader accessibility.

Despite good planning, certain tasks such as analysing the convergence behaviour and visualising trust propagation took longer than expected due to debugging and graph style according to my research objectives. So, decision making was crucial at this point of time. I had to learn to account for buffer time in future projects, particularly in testing and documentation phases. I completely utilised Microsoft Word for the report writing, which proved useful when reverting faulty changes. The ability to manage scope and expectations while delivering high quality work was one of the most valuable lessons learned from this dissertation phase.

9.4 Areas for Personal and Professional Development

This dissertation taught me technical depth and academic independence. The iterative cycle of reading, coding, failing, debugging and analysing helped me build persistence, endurance and self-direction. I learned how to search for the right papers rather than just recent ones, how to balance ideals models for my assumptions with practical constraints and how to write a high-quality report for both expert readers and new audiences.

Going forward I aim to expand my understanding of trust-based systems and decentralized protocol design, possibly applying them in domains like cloud security or peer reputation in federated learning systems. I am also interested in exploring machine learning for anomaly detection in such networks where trust scores could be dynamically learned from data instead of being manually updated. Areas where I could improve include automating result export

functions and validating my approach across multiple test cases. I also realised the significance of statistical validation, where future versions could benefit from more repetitions and confidence intervals to make results statistically noticeable.

On the personal side, this research has made me more confident in handling ambiguity, making design decisions, and justifying my approaches during discussions with my supervisor. I have learned to manage time, validation of literature, feedback iteration and reproducibility. But I need to improve confidence in statistical reasoning and alternative evaluation techniques. These learnings have extensively contributed for my personal and professional development.

References

1. **Poon, J. and Dryja, T. (2016)** *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. [Online] Available at: <https://lightning.network/lightning-network-paper.pdf> (Accessed: 23 August 2025).
2. **Corcoran, P. and Lewis, R. (2024)** ‘Path planning in payment channel networks with multi-party channels’, *Distributed Ledger Technologies*, ACM. doi: 10.1145/3702248.
3. **Seres, I.A., Gulyás, L., Nagy, D.A. and Burcsi, P. (2020)** ‘Topological analysis of Bitcoin’s Lightning Network’, *arXiv preprint*, arXiv:2006.00144. [Online] Available at: <https://arxiv.org/abs/2006.00144> (Accessed: 23 August 2025).
4. **Pérez-Solà, C., Delgado-Segura, S., Navarro-Arribas, G. and Herrera-Joancomartí, J. (2020)** ‘Lockdown: Balance availability attack against Lightning Network channels’, *Computer Communications*, 163, pp. 46–60. doi:10.1016/j.comcom.2020.09.003.
5. **Kamvar, S.D., Schlosser, M.T. and Garcia-Molina, H. (2003)** ‘The EigenTrust algorithm for reputation management in P2P networks’, in *Proceedings of the 12th International Conference on World Wide Web (WWW '03)*, Budapest, Hungary, 20–24 May. New York: ACM, pp. 640–651. doi:10.1145/775152.775242.
6. **Fan, X., Chen, Z. and Liu, L. (2012)** ‘EigenTrust++: Leveraging trusted friends for global trust evaluation in unstructured P2P networks’, *IEEE Transactions on Parallel and Distributed Systems*, 24(6), pp. 1127–1139. doi:10.1109/TPDS.2012.225.
7. **Aina, O. and Litchfield, A. (2012)** ‘A trust-based security mechanism for nomadic peer-to-peer networks’, *Journal of Network and Computer Applications*, 35(2), pp. 756–764. doi:10.1016/j.jnca.2011.11.004.
8. **BBC News (2025)** ‘Crypto heist: North Korea-linked hackers steal \$100m in tokens’. [Online] Available at: <https://www.bbc.co.uk/news/articles/c2kgndwwd7lo> (Accessed: 23 August 2025).
9. **Guasoni, P., Minca, A. and Sedunov, J. (2024)** ‘The Lightning Network: Topology, privacy, and economics’, *Journal of Financial Data Science*, 6(2), pp. 77–93. doi:10.3905/jfds.2024.1.081.

10. **Shikhelman, C. and Tikhomirov, S. (2022)** *Unjamming Lightning: A Systematic Approach*. Cryptology ePrint Archive, Paper 2022/1454. [Online] Available at: <https://eprint.iacr.org/2022/1454> (Accessed: 23 August 2025).
11. **Nakamoto, S. (2008)** *Bitcoin: A Peer-to-Peer Electronic Cash System*. [Online] Available at: <https://bitcoin.org/bitcoin.pdf> (Accessed: 19 August 2025).
12. **Tikhomirov, S., Moreno-Sanchez, P. and Maffei, M. (2020)** ‘A quantitative analysis of security, anonymity and scalability for the Lightning Network’, in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, pp. 48–56. doi:10.1109/EuroSPW51379.2020.00014.
13. **Bitcoin Optech (n.d.)** ‘Channel jamming attacks’. [Online] Available at: <https://bitcoinops.org/en/topics/channel-jamming-attacks/> (Accessed: 23 August 2025).
14. **Hagberg, A.A., Schult, D.A. and Swart, P.J. (2008)** ‘Exploring network structure, dynamics, and function using NetworkX’, in Varoquaux, G., Vaught, T. and Millman, J. (eds.) *Proceedings of the 7th Python in Science Conference (SciPy2008)*, Pasadena, CA, USA, 11–15 August. pp. 11–15.
15. **Xiong, L. and Liu, L. (2004)** ‘PeerTrust: Supporting reputation-based trust for peer-to-peer electronic communities’, *IEEE Transactions on Knowledge and Data Engineering*, 16(7), pp. 843–857. doi:10.1109/TKDE.2004.1318566.
16. **Zhou, R. and Hwang, K. (2007)** ‘PowerTrust: A robust and scalable reputation system for trusted peer-to-peer computing’, *IEEE Transactions on Parallel and Distributed Systems*, 18(4), pp. 460–473. doi:10.1109/TPDS.2007.1040.
17. **Wang, Y., Vassileva, J. and Zheng, Y. (2011)** ‘Trust-based peer-to-peer search with dynamic trust update’, *Future Generation Computer Systems*, 27(8), pp. 1096–1107. doi:10.1016/j.future.2011.03.006.
18. **Wei, W., Xu, F. and Jin, H. (2012)** ‘AntTrust: A trust model based on ant colony algorithm for peer-to-peer systems’, *Journal of Computer and System Sciences*, 78(1), pp. 39–55. doi:10.1016/j.jcss.2011.02.006.
19. **Zhang, J., Wang, C., Gong, S., Zhou, Y. and Yu, S. (2020)** ‘A survey on trust management for Internet of Things’, *Journal of Network and Computer Applications*, 123, pp. 1–16. doi:10.1016/j.jnca.2020.102889.
20. **Lin, X., Li, H., Liu, X., Yang, Y. and Shen, X. (2021)** ‘Trust management in distributed systems: A machine learning perspective’, *IEEE Network*, 35(1), pp. 250–257. doi:10.1109/MNET.011.2000321.